Appunti

Programmazione di dispositivi mobili e interfacce utente

Insegnamento di Programmazione di dispositivi mobili e interfacce utente

Appunti redatti da

Leonardo Malaventura

Università degli Studi di Urbino Carlo Bo
Laurea magistrale in Informatica e Innovazione Digitale



Anno accademico 2026–2027

Appunti di studio non ufficiali, in aggiornamento.

Indice

1 Applicazioni, programmi e processi . . . 1
- 1.1 Che cosa rende un programma un'applicazione . . . 1
- 1.2 Programma, eseguibile e processo . . . 1
- 1.3 Dal codice sorgente all'esecuzione . . . 2
- 1.4 Struttura dell'eseguibile e caricamento . . . 3
- 1.5 Librerie statiche e dinamiche . . . 3

2 La piattaforma applicativa e le sue interfacce . . . 4
- 2.1 Livelli di astrazione . . . 4
- 2.2 Componenti come black box: API e ABI . . . 5
- 2.3 Due estremi: sistemi embedded e browser . . . 5

3 Evoluzione delle piattaforme native . . . 6
- 3.1 Windows: dalle API native a WinRT e UWP . . . 6
- 3.2 Apple: Cocoa, AppKit, UIKit e SwiftUI . . . 6
- 3.3 Android: da Dalvik ad ART, da Java a Kotlin . . . 7

4 Il Web come piattaforma applicativa . . . 8
- 4.1 Dal documento ipertestuale all'interazione . . . 8
- 4.2 JavaScript e DOM: la pagina diventa modificabile . . . 8
- 4.3 Comunicazione asincrona con il server: XHR, Fetch e AJAX . . . 9
- 4.4 Indirizzare le risorse: struttura di un URL . . . 10
- 4.5 Siti multipagina e Single Page Application . . . 10
- 4.6 Rendering lato client e lato server . . . 11
- 4.7 Framework moderni e interfacce dichiarative . . . 11
- 4.8 Progressive Web Application . . . 12
- 4.9 Dal Web al desktop: Electron . . . 13

5 Sviluppo multipiattaforma, distribuzione e sicurezza . . . 14
- 5.1 Come scegliere una piattaforma . . . 14
- 5.2 Sviluppo nativo e multipiattaforma . . . 14
- 5.3 Il problema delle dipendenze nell'installazione tradizionale . . . 15
- 5.4 Pacchetti, manifest e permessi . . . 15
- 5.5 Sandboxing e modello dei permessi . . . 15

6 Il ciclo di vita di processi e applicazioni . . . 17
- 6.1 Perché è necessario distinguere i due cicli di vita . . . 17
- 6.2 Dal modello DOS ai sistemi multiprocesso . . . 17
- 6.3 Gli stati fondamentali di un processo . . . 17
- 6.4 Creazione e sostituzione dei processi su Unix . . . 18
- 6.5 Processi zombie e orfani . . . 19
- 6.6 Processi iniziali e servizi di sistema . . . 19
- 6.7 Applicazioni desktop e ciclo degli eventi . . . 19

7 Il modello di esecuzione delle applicazioni mobili . . . 21
- 7.1 Perché il mobile richiede un modello diverso . . . 21
- 7.2 Foreground, background, sospensione e terminazione . . . 21
- 7.3 Ciclo di vita dell'applicazione su iOS . . . 22
- 7.4 Ciclo di vita delle applicazioni UWP . . . 22

7.5 Android: componenti dell'applicazione e processi . . . 22
7.6 Ciclo di vita di una Activity Android . . . 23
7.7 Salvataggio e ripristino dello stato . . . 24

8 Dall'elaborazione differita alle interfacce interattive . . . 26
8.1 Perché studiare la storia delle interfacce . . . 26
8.2 Schede perforate e programmazione senza feedback immediato . . . 26
8.3 Batch processing: più lavori, poca interazione . . . 26
8.4 Console, teletype e interazione a comandi . . . 26
8.5 Time sharing e terminali video . . . 27
8.6 Che cosa cambia per l'utente: feedback e carico cognitivo . . . 27
8.7 Engelbart e la "Mother of All Demos" . . . 27
8.8 Dalle prime GUI al paradigma WIMP . . . 28
8.9 Metafora della scrivania e modello WYSIWYG . . . 28
8.10 X Window System: la GUI non coincide con il computer locale . . . 29
8.11 Verso le GUI moderne . . . 29

9 Linguaggi visivi delle interfacce grafiche . . . 30
9.1 Skeuomorfismo: riconoscere oggetti familiari . . . 30
9.2 Flat design: essenzialità e riconoscibilità delle azioni . . . 30
9.3 Material Design e il ruolo del movimento . . . 30
9.4 Fluent Design, glassmorfismo e Liquid Glass . . . 31

10 Rasterizzazione, densità e dimensionamento delle interfacce . . . 32
10.1 Dal tubo catodico ai pixel fisici degli schermi moderni . . . 32
10.2 Rasterizzazione, aliasing e antialiasing . . . 33
10.3 Dal punto tipografico alla densità dello schermo . . . 33
10.4 Pixel logici, fattore di scala e indipendenza dal dispositivo . . . 34
10.5 DPI awareness: chi gestisce l'adattamento della UI? . . . 35

11 Un esempio trasversale: la stessa applicazione su piattaforme diverse 37

12 Laboratorio: introduzione a Flutter e Dart . . . 38
12.1 Flutter: scopo e modello di esecuzione . . . 38
12.2 Dart: ruolo e caratteristiche generali . . . 38
12.3 Organizzazione di un progetto Dart da console . . . 39

13 Fondamenti del linguaggio Dart . . . 40
13.1 Tipi, inferenza e assegnazione . . . 40
13.2 `var`, `final`, `const` e `late` . . . 40
13.3 Null safety e operatori di accesso sicuro . . . 41
13.4 Funzioni: parametri posizionali e nominati . . . 41
13.5 Condizioni, iterazioni ed espressioni condizionali . . . 42
13.6 Interpolazione di stringhe e collezioni . . . 43
13.7 Prima esercitazione: lettura del programma `language_tour` . . . 44
13.8 Classi, costruttori e incapsulamento . . . 46
13.9 Esempio completo: modellare uno studente . . . 46

Capitolo 1

Applicazioni, programmi e processi

1.1 Che cosa rende un programma un'applicazione

Un'**applicazione** è un sistema software che realizza funzionalità rivolte a un utilizzatore. Per capire in che senso sia un programma, occorre distinguere tre domande: *che cosa deve fare*, *come viene eseguita* e *come l'utente vi interagisce*. Le risposte non coincidono necessariamente con lo stesso pezzo di software.

Si consideri una piccola applicazione per prendere appunti. La sua **logica applicativa** gestisce operazioni come creare una nota, modificarne il testo, cercarla e salvarla. Un'**interfaccia utente** (*User Interface*, UI) rende disponibili queste funzioni, ad esempio tramite pulsanti e campi di testo. Un'implementazione potrebbe usare una finestra desktop, una schermata mobile o una pagina Web: le differenze nell'interazione non implicano necessariamente una differenza nelle regole secondo cui le note sono gestite.

Questo spiega perché diciamo che la UI è uno **strato concettualmente distinguibile** dalla logica. Non è obbligatorio che i due strati siano fisicamente separati in processi o file diversi: in un programma piccolo possono essere mescolati. La separazione è utile perché rende più semplice riusare la logica, cambiare la tecnologia con cui si presenta l'applicazione e verificare il comportamento senza interagire manualmente con una finestra.

Un'interfaccia, inoltre, non è necessariamente grafica e non è necessariamente destinata a una persona. Un'applicazione può offrire una riga di comando, un'API per altri programmi oppure comunicare attraverso file o messaggi di rete. Una **interfaccia software** definisce come due componenti comunicano; la **UI** è il caso particolare in cui uno degli interlocutori è l'utente.

È infine utile distinguere il **software applicativo**, orientato alle attività dell'utente, dal **software di sistema**, che fornisce infrastruttura e servizi agli altri programmi. Non è una classificazione assoluta: un browser, per esempio, è un'applicazione usata dall'utente ma costituisce anche una piattaforma per eseguire altre applicazioni Web. Questa doppia natura sarà importante più avanti.

1.2 Programma, eseguibile e processo

Un **programma** descrive una computazione: le istruzioni e le informazioni necessarie per realizzare un comportamento. È una nozione **statica**. La rappresentazione del programma può essere il sorgente, un codice intermedio oppure un file eseguibile nativo.

Un **eseguibile** è una rappresentazione del programma preparata per un ambiente di esecuzione. L'eseguibile nativo è riconosciuto dal sistema operativo e contiene codice macchina e metadati di caricamento. Non tutte le applicazioni sono però distribuite in questa forma: una pagina JavaScript, ad esempio, è eseguita tramite l'ambiente offerto dal browser.

Quando il sistema avvia un programma, viene creato un **processo**, cioè un'**istanza del programma in esecuzione**. Il processo non è semplicemente "il file che sta girando": comprende uno stato che muta nel tempo. Tra gli elementi rilevanti troviamo lo **spazio di indirizzamento**, lo stack dei thread, la memoria dinamica, i descrittori di file e le

altre risorse aperte, oltre alle informazioni che il sistema operativo usa per pianificarne l'esecuzione, fra cui un identificatore chiamato *Process ID* (PID).

La distinzione emerge con un esempio. Il file di un editor è uno solo; se lo avviamo due volte, potremmo ottenere due processi, ciascuno con finestre, memoria e documenti aperti propri. Viceversa una sola applicazione può usare **più processi**, per esempio separando l'interfaccia da un servizio ausiliario. L'applicazione è pertanto l'**unità funzionale**, mentre il processo delimita soprattutto **risorse e spazio di memoria** messi a disposizione dell'esecuzione.

Per capire che cosa esegua materialmente la CPU occorre distinguere anche il **thread**. Un processo contiene almeno un thread di esecuzione e può contenerne diversi: questi condividono lo spazio di indirizzamento e le risorse del processo, ma possiedono un proprio flusso di istruzioni, uno stack e lo stato dei registri. Nei comuni sistemi operativi moderni è il **thread** l'entità che lo scheduler assegna alla CPU. Per questo una singola applicazione può avere più attività in corso nello stesso processo senza richiedere necessariamente un processo distinto per ciascuna attività.

Questa distinzione anticipa un tema fondamentale dei dispositivi mobili: un'applicazione può continuare a essere installata e conservare i propri dati anche quando il sistema ha *terminato il suo processo*.

1.3 Dal codice sorgente all'esecuzione

Il **codice sorgente** è la rappresentazione del programma scritta in un linguaggio di programmazione. La CPU, invece, esegue le istruzioni del proprio **linguaggio macchina** (*instruction set*): non sa eseguire direttamente un testo Java, C o Kotlin. Serve quindi una catena di traduzione o un ambiente che interpreti il codice.

Compilazione nativa. Nel modello tradizionale il **compilatore** analizza il sorgente e produce uno o più moduli di **codice oggetto**. Il **linker** combina i moduli e risolve i riferimenti tra funzioni, variabili e librerie, producendo un eseguibile o un altro artefatto binario:

$$\text{sorgente} \longrightarrow \text{compilatore} \longrightarrow \text{moduli oggetto} \longrightarrow \text{linker} \longrightarrow \text{eseguibile}.$$

Per esempio, un programma C che richiama una funzione definita in un altro file deve poter collegare la chiamata alla corretta implementazione. Il compilatore gestisce la traduzione dei singoli moduli; il linker risolve le connessioni necessarie. **L'eseguibile non è ancora un processo**: nasce come file e diventa una computazione attiva solo quando è caricato e avviato.

Interpretazione. Un **interprete** è un programma che legge una rappresentazione delle istruzioni e ne realizza il comportamento. Nella forma più semplice legge ed esegue il programma passo dopo passo, ma un interprete reale può anche analizzare, precompilare e ottimizzare il codice. In questo modello è l'interprete stesso a essere avviato come programma nativo dal sistema operativo.

Codice intermedio e runtime. Una terza possibilità consiste nel tradurre il sorgente in **bytecode**, un formato intermedio pensato per un ambiente di esecuzione anziché per una singola CPU. Un **runtime** compatibile può interpretarlo oppure trasformarlo in codice macchina. Per esempio il codice Android viene distribuito in una rappresentazione DEX che il runtime della piattaforma sa gestire. Il vantaggio è separare, almeno in parte, la rappresentazione del programma dall'hardware; il prezzo è la necessità di un ambiente compatibile sul dispositivo.

Queste soluzioni **non sono categorie esclusive**. Un runtime può eseguire codice attraverso interpretazione, *Just-In-Time* (JIT: compilazione durante l'esecuzione) e *Ahead-Of-Time* (AOT: compilazione anticipata). La distinzione utile non è quindi "linguaggio compilato contro linguaggio interpretato", ma **quale trasformazione viene effettuata, quando e da chi**.

1.4 Struttura dell'eseguibile e caricamento

Un eseguibile nativo non è una sequenza casuale di byte. Segue un **formato binario** che il sistema operativo è in grado di interpretare. In esso possiamo trovare un **header**, che indica fra le altre cose il tipo di file e l'architettura prevista; sezioni con **istruzioni macchina** e **dati**; informazioni per le librerie e, quando previste o conservate, tabelle di simboli e dati utili al collegamento o al debugging.

Una informazione fondamentale è l'**entry point**, cioè il punto a partire dal quale inizia l'esecuzione del codice caricato. Non coincide necessariamente con la funzione `main()` scritta dal programmatore: prima possono intervenire routine di avvio e inizializzazione delle librerie.

Che cosa accade quando l'utente apre un programma? In un caso tipico:

1. il sistema operativo riceve la richiesta di avvio;
2. crea le strutture necessarie per un processo e prepara il suo spazio di indirizzamento;
3. il **loader** mappa il file eseguibile in memoria e predispone eventuali librerie dinamiche;
4. il codice di inizializzazione prepara l'ambiente e il controllo raggiunge l'entry point;
5. i thread eseguono istruzioni e interagiscono con i servizi del sistema operativo;
6. alla terminazione vengono rilasciate le risorse e reso disponibile lo **stato d'uscita** (*exit status*).

I passaggi effettivi dipendono dal sistema e dal formato dell'eseguibile. Soprattutto, **il loader non va confuso con il compilatore**: il primo prepara all'esecuzione un programma già prodotto; il secondo trasforma il codice sorgente.

1.5 Librerie statiche e dinamiche

Un'applicazione non deve implementare da zero tutte le operazioni che utilizza. Le **librerie** contengono codice riutilizzabile che fornisce funzionalità, come la manipolazione di stringhe o la comunicazione con il sistema operativo. La questione è *quando* e *come* collegare quel codice all'applicazione.

Con il **linking statico** il codice di libreria necessario viene incluso nell'eseguibile in fase di costruzione. Il programma risultante può avere meno dipendenze esterne a runtime, ma tende a occupare più spazio e può duplicare lo stesso codice fra applicazioni diverse.

Con il **linking dinamico** il codice di alcune librerie rimane separato. Il sistema lo mette a disposizione durante il caricamento o l'esecuzione. Più programmi possono condividere librerie compatibili; aggiornare una libreria può inoltre aggiornare il comportamento dei programmi che la utilizzano. D'altra parte, diventa necessario controllarne presenza e compatibilità.

Si immagini che due applicazioni dipendano da versioni incompatibili della stessa libreria: se entrambe chiedono una versione "globale", installarne una può compromettere l'altra. Questa situazione anticipa il problema della **gestione delle dipendenze**, che motiva i pacchetti applicativi trattati più avanti. Non esiste quindi una categoria sempre "più sicura" o "più robusta": la scelta è un compromesso tra distribuzione, riuso, dimensione, compatibilità e aggiornabilità.

Capitolo 2

La piattaforma applicativa e le sue interfacce

2.1 Livelli di astrazione

Un programma moderno raramente interagisce direttamente con i registri e le periferiche del dispositivo. Utilizza invece una **piattaforma applicativa**, cioè un insieme di servizi, convenzioni e strumenti che gli permettono di essere eseguito e di accedere alle risorse della macchina. La piattaforma evita allo sviluppatore di risolvere ripetutamente problemi di basso livello.



Figura 1: Livelli concettuali della piattaforma: non tutte le tecnologie presentano questi strati come elementi separati.

Hardware e sistema operativo. L'hardware comprende CPU, memoria e periferiche; il sistema operativo gestisce risorse, processi, protezione, file e accesso ai dispositivi. Invece di controllare direttamente una fotocamera, un'applicazione può chiedere al sistema un servizio attraverso le API previste.

Runtime. È l'ambiente che rende possibile l'esecuzione di un determinato tipo di programma. A seconda della piattaforma può includere caricamento di moduli, gestione della memoria, garbage collection e interpretazione o compilazione di codice intermedio. Il runtime non è semplicemente "un altro sistema operativo": opera in un ambiente che gli fornisce già servizi e risorse.

Framework. Fornisce al programmatore componenti, convenzioni e spesso un modello di controllo dell'applicazione: finestre, navigazione, eventi, widget e così via. Il suo scopo non è principalmente tradurre istruzioni per la CPU, ma **semplificare la costruzione del software**. Un framework UI, per esempio, può sapere come creare un pulsante, reagire a un tocco e aggiornare una schermata.

Consideriamo un pulsante "Salva" nella nostra app di note. Il **framework** gestisce la visualizzazione e l'evento del tocco; la **logica applicativa** decide che cosa registrare; il **runtime** supporta l'esecuzione del codice; il **sistema operativo** fornisce servizi di memoria e archiviazione; l'**hardware** realizza materialmente le operazioni. Questa catena non impone che i componenti siano contenuti gli uni negli altri come cartelle: è uno **schema di responsabilità**, non di impacchettamento.

Infatti alcune dipendenze sono già installate nella piattaforma, altre vengono distribuite insieme all'applicazione. La scelta influenza dimensioni del pacchetto, portabilità e requisiti

d'installazione, ma non cambia la distinzione concettuale tra gli strati.

2.2 Componenti come black box: API e ABI

I diversi strati possono essere sviluppati e modificati indipendentemente solo se è chiaro **come comunicano**. Un componente si può trattare come una **black box**: l'utilizzatore vede un insieme di operazioni e comportamenti previsti, senza dover conoscere il codice che li realizza. Un'implementazione può essere sostituita finché mantiene il contratto richiesto dagli utilizzatori.

Una **API** (*Application Programming Interface*) descrive quel contratto dal punto di vista del software che richiede un servizio. Può specificare nomi di funzioni, tipi dei parametri, valori restituiti, errori e significato dell'operazione. Se una funzione è dichiarata `salvaNota(testo)`, per utilizzarla occorre sapere che cosa accetta, che cosa promette di fare e quali errori possono verificarsi: non come gestisce concretamente il disco. Il termine API non è limitato alle API Web: esistono API di libreria e API del sistema operativo.

Una **ABI** (*Application Binary Interface*) descrive invece le **convenzioni di interoperabilità a livello binario**. Quando una funzione compilata ne chiama un'altra, occorre sapere come passare i parametri (registri o stack), come rappresentare i tipi, chi ripristina determinati registri e in quale forma organizzare i binari. Queste convenzioni dipendono in genere dall'architettura hardware, dal sistema operativo e dalla toolchain.

La differenza diventa concreta immaginando due librerie con la stessa firma di funzione nel codice sorgente. Se sono state compilate con convenzioni binarie incompatibili, non possono necessariamente essere collegate e chiamate correttamente, anche se l'**API sembra identica**. In altre parole, la compatibilità a livello di sorgente non garantisce la compatibilità a livello di binario.

	API	**ABI**
Domanda	Quale servizio posso richiedere e con quale significato?	Come devono cooperare le istruzioni già compilate?
Livello	Contratto software, spesso visibile nel sorgente	Convenzioni binarie e di chiamata
Esempio	Firma di `apriFile()`	Posizione dei parametri nei registri o nello stack

I due livelli sono complementari: un'applicazione può usare un'API senza conoscere i dettagli della relativa ABI, ma un collegamento fra moduli nativi deve rispettare anche le convenzioni binarie.

2.3 Due estremi: sistemi embedded e browser

In un **sistema embedded** semplice l'applicazione può eseguire un ciclo principale (*application loop*) sopra un **Hardware Abstraction Layer** (HAL), che nasconde alcuni dettagli dei dispositivi. In taluni casi non è presente un sistema operativo general purpose; in altri si usa un **Real-Time Operating System** (RTOS), nel quale il rispetto dei vincoli temporali è parte della correttezza. La presenza di un RTOS, tuttavia, non esclude l'esistenza di API o convenzioni binarie.

All'altro estremo, un'**applicazione Web** viene eseguita dentro un **browser**, che costituisce a sua volta una piattaforma sopra il sistema operativo. Il browser offre il motore di rendering, l'ambiente JavaScript e numerose Web API. La disponibilità delle funzioni e il loro comportamento possono variare tra browser e sistemi: *multipiattaforma* non significa sempre *identico su ogni dispositivo*.

Capitolo 3

Evoluzione delle piattaforme native

Le piattaforme Windows, Apple e Android mostrano come la programmazione si sia spostata dalle chiamate di basso livello a framework e modelli applicativi più strutturati. Le date servono come riferimenti storici; per lo sviluppo è soprattutto importante capire **quale astrazione ciascuna evoluzione abbia introdotto**.

3.1 Windows: dalle API native a WinRT e UWP

Alcune tappe sono **Windows 1.0** (1985), **Windows 3.1** (1992), **Windows 95** (1995) e **Windows XP** (2001). Le prime versioni hanno utilizzato API a 16 bit (**Win16**); la famiglia **Win32**, associata anche a Windows NT, ha fornito le principali interfacce native per finestre, file, processi e altri servizi. In ambiente a 64 bit, le API Windows hanno mantenuto una notevole continuità, con adattamenti alle convenzioni e ai tipi dell'architettura.

Con Windows 8 viene introdotto **Windows Runtime** (**WinRT**), un modello di API orientato agli oggetti utilizzabile da diversi linguaggi attraverso apposite *language projections*. Questo non rende WinRT indipendente dal sistema operativo: rimane una tecnologia Windows. WinRT favorisce anche un modello moderno di applicazione, con isolamento e ampio impiego delle operazioni asincrone.

Un'operazione **sincrona bloccante** impedisce al flusso chiamante di proseguire finché non produce il risultato. Se avviene sul thread che gestisce la UI, un'operazione lenta di rete o di accesso ai dati può rendere l'interfaccia temporaneamente non responsiva. Una **operazione asincrona** consente invece di avviare il lavoro e riprendere l'esecuzione utile mentre il risultato non è ancora disponibile. Asincronia e parallelismo non sono sinonimi: l'asincronia organizza l'attesa senza imporre che ogni operazione sia eseguita da un thread distinto.

Con Windows 10 compare **Universal Windows Platform** (**UWP**), pensata per offrire una base di API e un modello applicativo condivisi tra famiglie di dispositivi Windows. Le applicazioni di questo modello sono spesso chiamate *Universal Windows Applications*. Il suo ciclo di vita gestito dal sistema sarà ripreso nella sezione dedicata.

3.2 Apple: Cocoa, AppKit, UIKit e SwiftUI

Il Macintosh originale risale al **1984**. Il successivo ambiente **Cocoa**, derivato storicamente dalle tecnologie NeXT, è associato allo sviluppo nativo per macOS. **AppKit** fornisce elementi e meccanismi per finestre, menu, controlli ed eventi delle interfacce desktop. Le API sono storicamente legate a **Objective-C**; **Swift** è stato poi introdotto come moderno linguaggio capace di interoperare con questo ecosistema.

Per iOS e iPadOS si è affermato **Cocoa Touch**, con **UIKit** come framework fondamentale per interfacce e gestione di molte interazioni. **SwiftUI** introduce un diverso stile, **dichiarativo**: la vista viene descritta in funzione dello stato invece di essere modificata esclusivamente tramite sequenze esplicite di comandi. AppKit, UIKit e SwiftUI non vanno considerati tre sistemi operativi, ma strumenti e modelli di UI all'interno delle piattaforme Apple.

Mac Catalyst consente di riutilizzare una codebase UIKit per realizzare una versione macOS di un'app progettata per iPad. La condivisione del codice non elimina automaticamente la necessità di adattare l'esperienza ai diversi dispositivi.

3.3 Android: da Dalvik ad ART, da Java a Kotlin

Android utilizza una rappresentazione intermedia del codice applicativo nel formato **DEX**. Le prime versioni eseguivano questo codice tramite il runtime **Dalvik**. **Android Runtime** (**ART**) è stato introdotto come alternativa sperimentale in Android 4.4 ed è diventato il runtime principale da Android 5.0: il suo ruolo è eseguire e ottimizzare il codice dell'applicazione.

Storicamente le API Android venivano utilizzate soprattutto con **Java**; oggi **Kotlin** è centrale nello sviluppo Android. Bisogna distinguere *il linguaggio in cui si scrive l'app* dal *runtime che ne esegue il codice*. Il passaggio da Java a Kotlin non coincide con il passaggio da Dalvik ad ART.

Capitolo 4

Il Web come piattaforma applicativa

4.1 Dal documento ipertestuale all'interazione

Per capire perché oggi un browser può eseguire applicazioni complesse conviene partire dal modello originario del Web. In un sito **documentale**, il browser richiede una risorsa tramite HTTP, riceve un documento HTML e lo visualizza. Quando l'utente segue un collegamento, il browser richiede un altro documento. L'interazione ha quindi il ritmo di una **navigazione tra pagine**, con una relazione semplice tra URL, documento e schermata.

L'espressione **Web 2.0** indica un cambiamento nel modo di usare il Web: contenuti aggiornabili, interazione più ricca e maggiore partecipazione degli utenti. Non definisce una versione ufficiale di HTTP o un unico stack tecnologico. Per esempio, una pagina che permette di modificare una nota direttamente mentre la si consulta offre un'interazione diversa da un documento statico da leggere.

In passato una possibilità era inserire nella pagina un ambiente software separato, come le **Java applet** o **Flash**. Queste tecnologie potevano svolgere attività interattive che i browser dell'epoca non supportavano direttamente, ma richiedevano plugin o runtime aggiuntivi, con problemi di compatibilità, aggiornamento e superficie di attacco. Non è rigoroso dire che non avessero *alcun* controllo di sicurezza: la questione era che il modello di sicurezza e integrazione risultava complesso e non uniforme.

Si è affermata un'altra soluzione: far crescere le **capacità native del browser**, tramite JavaScript e API standard. Il browser è così passato dall'essere prevalentemente un visualizzatore di documenti a diventare una **piattaforma applicativa**. Questo percorso spiega le tecnologie che incontreremo: DOM per modificare la UI, Fetch per recuperare dati, strumenti per la navigazione e framework per coordinare il tutto.

4.2 JavaScript e DOM: la pagina diventa modificabile

HTML descrive la struttura del documento, CSS la sua presentazione, mentre **JavaScript** consente di reagire a eventi e modificare l'interfaccia nel tempo. Per renderlo possibile, il browser espone il **Document Object Model** (**DOM**), cioè una rappresentazione ad albero della pagina accessibile da codice.

Si consideri la sequenza seguente:

```
<p id="stato">Nota non salvata</p>
<button id="salva">Salva</button>
```

Un programma JavaScript può associare un'azione al pulsante e aggiornare il testo di un elemento:

```
const stato = document.getElementById("stato");
document.getElementById("salva").addEventListener("click", () => {
    stato.textContent = "Salvataggio richiesto";
});
```

Il click produce un **evento**; JavaScript modifica il DOM; il browser esegue gli aggiornamenti necessari a visualizzare il nuovo contenuto. In questo esempio cambia solo il testo di un

nodo, senza richiedere al server una nuova pagina HTML. Non bisogna confondere il **DOM**, che è una struttura di oggetti in memoria, con il **file HTML**, che è una rappresentazione testuale iniziale: dopo l'esecuzione di JavaScript i due contenuti possono differire.

Il DOM rende quindi la pagina *modificabile*, ma non dice ancora come ottenere dati aggiornati da un server. Per risolvere il secondo problema servono le API per la comunicazione di rete.

4.3 Comunicazione asincrona con il server: XHR, Fetch e AJAX

Se la nostra app di note mostrasse l'elenco delle note ricevute dal server, un modello documentale potrebbe richiedere un'intera pagina HTML ogni volta che ne aggiungiamo una. Si tratta di un approccio corretto, ma per una UI molto interattiva è spesso inutile ricostruire tutto il documento quando cambiano soltanto pochi dati.

Le API **XMLHttpRequest** (XHR) e **Fetch** permettono a JavaScript di effettuare richieste HTTP e gestirne le risposte separatamente dalla navigazione principale. Con **AJAX** (*Asynchronous JavaScript and XML*) si indica l'**approccio** che combina comunicazione con il server, codice JavaScript e aggiornamenti dell'interfaccia. **AJAX non è un framework**: il nome è storico e non implica che la risposta debba essere in XML; JSON è oggi molto frequente.

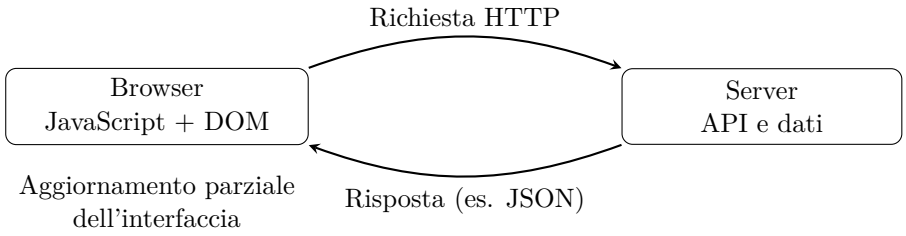


Figura 2: Richiesta di dati dal browser e successivo aggiornamento della UI, senza obbligo di caricare un nuovo documento.

Un esempio con Fetch è:

```
async function caricaNote() {
    const risposta = await fetch("/api/note");
    if (!risposta.ok) throw new Error("Richiesta fallita");
    const note = await risposta.json();
    return note;
}
```

`fetch()` avvia una richiesta; `await` sospende l'esecuzione della funzione asincrona fino alla disponibilità del risultato, **senza bloccare l'intero thread come farebbe un'attesa sincrona bloccante**. I dati restituiti possono quindi essere usati per aggiornare una porzione della UI.

È importante distinguere **asincronia** da **parallelismo**: la prima permette di gestire l'attesa di un risultato senza fermare necessariamente le altre attività; il secondo riguarda attività che procedono effettivamente in contemporanea su risorse di esecuzione diverse. Anche una richiesta asincrona resta soggetta a errori di rete, permessi e politiche di origine: non è una chiamata priva di vincoli.

4.4 Indirizzare le risorse: struttura di un URL

Un **URL** (*Uniform Resource Locator*) specifica dove e come raggiungere una risorsa. Si consideri:

```
https://example.org/path?search=ABC
```

Le parti principali sono:

- `https`: lo **schema**, che indica il protocollo di comunicazione;
- `example.org`: il **nome dell'host** a cui rivolgersi;
- `/path`: il **percorso** della risorsa;
- `search=ABC`: un parametro della **query string**, introdotta da `?`.

La query fornisce informazioni aggiuntive alla richiesta, ad esempio un criterio di ricerca. Non è una parte dell'indirizzo che debba necessariamente identificare un file fisico: la sua interpretazione dipende dall'applicazione server.

4.5 Siti multipagina e Single Page Application

Le richieste AJAX hanno aperto la strada a due modi distinti di organizzare la navigazione nel Web. La differenza non è semplicemente "sito vecchio contro sito moderno", ma **chi costruisce la schermata e che cosa succede quando cambia la vista**.

In un sito **multipagina** (*Multi-Page Application*, MPA) una navigazione tradizionale richiede un nuovo documento HTML al server. Per esempio, passando dalla lista delle note alla pagina di una singola nota, il browser può caricare una nuova risorsa e costruire un nuovo documento. Questo modello sfrutta direttamente meccanismi consolidati del Web: URL, cronologia, collegamenti, semantica dei documenti e navigazione standard.

In una **Single Page Application** (**SPA**), invece, il browser carica inizialmente il documento dell'applicazione e il codice JavaScript necessario per gestirne l'interfaccia. La navigazione interna può cambiare **vista** senza ricaricare un nuovo documento: passare dalla lista delle note all'editor consiste nell'aggiornare lo stato dell'app e gli elementi visualizzati. Le richieste successive al server possono trasferire soprattutto dati, spesso JSON, ma non è un vincolo universale: una SPA può ricevere anche HTML o utilizzare altre forme di comunicazione.

	Navigazione multipagina	**Navigazione SPA**
Cambio vista	Di norma nuovo documento	Di norma aggiornamento della UI senza nuovo documento
Ruolo di JS	Facoltativo per la navigazione di base	Centrale nella navigazione client-side
Vantaggio tipico	Meccanismi Web nativi e documento pronto	Transizioni e interazioni continue
Responsabilità	Parte gestita dalla navigazione del browser	Il codice deve coordinare routing, stato e aggiornamenti

La SPA offre esperienze fluide, ma **non è automaticamente più veloce**: può richiedere molti script all'avvio e gestire per lungo tempo uno stato complesso, con possibili errori e consumi di memoria. Deve inoltre curare navigazione, cronologia, accessibilità e indicizzazione dei contenuti; strumenti moderni possono affrontare questi problemi, ma non

scompaiono da soli. Anche un sito multipagina può utilizzare JavaScript e AJAX: i modelli non sono opposti assoluti.

Il punto centrale è che la SPA **trasferisce al client una parte maggiore della responsabilità applicativa**, non che renda superflui i principi del Web.

4.6 Rendering lato client e lato server

La distinzione SPA/MPA riguarda soprattutto la navigazione; un'altra questione è **dove e quando viene prodotto l'HTML** da mostrare. Confondere queste due dimensioni porta a classificazioni sbagliate.

- **Client-Side Rendering (CSR)**: gran parte della vista viene costruita nel browser da JavaScript. Il server può inviare un documento iniziale essenziale e risorse JS; il client recupera poi gli eventuali dati e costruisce la UI.
- **Server-Side Rendering (SSR)**: il server genera HTML per una richiesta. Il browser può mostrarne subito il contenuto, e il codice client può successivamente renderlo interattivo. Il server *può* personalizzare la risposta, ma non è obbligato a farlo.
- **Static Site Generation (SSG)**: l'HTML viene generato in una fase precedente alle singole richieste, per esempio durante la build. La risposta può essere servita come risorsa statica. Non è corretto identificare SSG con "un sito che non cambia mai": può cambiare quando viene rigenerato e JavaScript può comunque renderne dinamiche alcune parti.
- **Rendering ibrido**: uno stesso progetto può applicare strategie diverse a pagine e componenti differenti.

Immaginiamo un servizio di note con una pagina pubblica che spiega il prodotto e un editor privato. La pagina pubblica potrebbe essere **generata staticamente**, perché è uguale per molti visitatori. La lista delle note personali potrebbe essere **renderizzata dal server** per mostrare immediatamente i dati dell'utente autenticato. L'editor potrebbe utilizzare **rendering e aggiornamenti lato client** mentre l'utente scrive. È una scelta architetturale guidata dalle esigenze, non un obbligo di scegliere una sigla per tutta l'app.

Va inoltre distinta la **generazione iniziale** della pagina dalle modifiche che avvengono **dopo** il caricamento. Una pagina SSR può diventare interattiva attraverso JavaScript; una pagina statica può recuperare dati tramite Fetch. Perciò "SSR = personalizzato" e "SSG = non personalizzabile" sono scorciatoie fuorvianti: ciò che cambia è **il momento della generazione dell'HTML**, non l'intera capacità di interazione del sito.

4.7 Framework moderni e interfacce dichiarative

Quando una pagina contiene pochi elementi, modificare manualmente il DOM può essere semplice. In un'applicazione articolata, però, la UI dipende da numerose informazioni: utente autenticato, nota selezionata, stato della sincronizzazione, eventuali errori e molte altre. Gestire manualmente ogni aggiornamento rischia di creare incoerenze.

Framework e librerie come **React**, **Angular** e **Next.js** aiutano a organizzare questa complessità. Non sono tutti lo stesso genere di strumento: React è principalmente una libreria per UI; Next.js aggiunge un framework applicativo con routing e strategie di rendering. Un principio comune a diversi strumenti moderni è descrivere l'interfaccia in forma **dichiarativa**, cioè come funzione dello stato:

$$\text{UI} = f(\text{stato}).$$

Si immagini di rappresentare lo stato della nostra applicazione come `notaSelezionata`, `testo` e `salvataggioInCorso`. Se `salvataggioInCorso` diventa vero, la vista deve mostrare un indicatore di caricamento e disabilitare temporaneamente il pulsante di salvataggio. Nell'approccio **imperativo** lo sviluppatore comanda direttamente queste modifiche ai componenti UI; in quello **dichiarativo** descrive quale vista corrisponde allo stato e il framework aggiorna concretamente gli elementi necessari.

L'equazione è un **modello concettuale**, non la pretesa che ogni framework calcoli matematicamente una funzione pura: nella pratica esistono effetti collaterali, operazioni di rete e meccanismi diversi di aggiornamento. La distinzione è utile perché il programmatore si concentra sul **risultato visivo coerente con lo stato**, anziché sul dettaglio di ogni modifica al DOM.

Il browser offre inoltre API come **Canvas** per disegnare grafica, **WebRTC** per comunicazioni multimediali e dati in tempo reale e **WebAssembly** per eseguire moduli in un formato binario portabile. Insieme alle normali Web API, queste capacità rendono possibili giochi, IDE, editor di testo e altre applicazioni complesse all'interno del browser.

4.8 Progressive Web Application

Una **Progressive Web Application** (**PWA**) usa funzionalità della piattaforma Web per offrire comportamenti normalmente associati a un'applicazione installata: ad esempio un'icona dedicata, l'avvio in una finestra autonoma e, se realizzata, la possibilità di funzionare anche senza connessione.

Una PWA **non è una via di mezzo obbligatoria fra SPA e MPA**: questa classificazione riguarda la navigazione. La qualifica PWA riguarda invece **modalità di distribuzione e capacità** offerte dall'app Web. Una SPA può essere PWA, ma può esserlo anche un sito multipagina. Si tratta di dimensioni diverse.

Due meccanismi ricorrenti sono:

- il **Web App Manifest**, un file JSON che descrive metadati come nome, icone, URL iniziale e preferenze di visualizzazione, utili per l'esperienza di installazione;
- il **Service Worker**, uno script eseguito separatamente dalla pagina, privo di accesso diretto al DOM, che può ricevere eventi e intercettare richieste delle pagine sotto il suo controllo.

Supponiamo che l'utente apra l'app di note in treno senza copertura. Uno sviluppatore può progettare una strategia in cui il Service Worker serve dalla **cache** le risorse dell'interfaccia già salvate, mentre l'app legge da un archivio locale le note disponibili offline. Quando torna la rete, l'app può tentare una sincronizzazione. La presenza del Service Worker non rende automaticamente disponibili tutti i dati: **offline è una funzionalità che va progettata**, anche per risolvere conflitti tra modifiche concorrenti.

Il Service Worker **non rimane sempre vivo**. Il browser può attivarlo per elaborare un evento e terminarlo quando non serve: non può essere usato come un processo illimitato in background. Funzionalità come notifiche push, sincronizzazione, file system e accesso all'hardware dipendono da API, autorizzazioni e supporto dei singoli browser e sistemi operativi.

Le PWA mostrano un punto generale: **una piattaforma Web può esporre funzionalità simili a quelle native**, ma l'app opera comunque nei limiti del modello di sicurezza e del ciclo di vita gestiti dal browser.

4.9 Dal Web al desktop: Electron

Un'applicazione Web può essere trasformata in un'app desktop distribuendola insieme a un ambiente capace di visualizzarla e interagire con il sistema operativo. **Electron** utilizza **Chromium** per il rendering e **Node.js** per le funzionalità lato processo principale.

Il suo modello comprende un **main process**, che gestisce applicazione e finestre, e processi **renderer**, che visualizzano i contenuti Web. La comunicazione tra questi processi avviene attraverso meccanismi di **Inter-Process Communication** (IPC), con particolare attenzione alla separazione tra codice della pagina e capacità privilegiate.

Esempi noti di applicazioni Electron comprendono **Visual Studio Code** e **Discord**. Il vantaggio è il riuso di tecnologie Web su sistemi diversi e la disponibilità di API desktop. Uno svantaggio è che ciascuna applicazione distribuisce parte consistente del proprio motore, con costi in termini di spazio e memoria. Un'errata esposizione di capacità Node.js a contenuti non attendibili può inoltre introdurre rischi di sicurezza: la separazione dei processi e dei privilegi è fondamentale.

Capitolo 5

Sviluppo multipiattaforma, distribuzione e sicurezza

5.1 Come scegliere una piattaforma

Scegliere la piattaforma per un'applicazione significa bilanciare vincoli tecnici e obiettivi del progetto, non semplicemente individuare il sistema più diffuso. I principali criteri comprendono:

- **pubblico e distribuzione**: quali dispositivi usano gli utenti e attraverso quali canali riceveranno l'applicazione;
- **esperienza d'uso**: convenzioni grafiche, accessibilità e comportamenti attesi dall'utente;
- **integrazione**: accesso a sensori, fotocamera, servizi del sistema e software preesistente;
- **sicurezza e protezione dei dati**: autorizzazioni, isolamento, policy della piattaforma e trattamento delle informazioni;
- **sviluppo e manutenzione**: competenze disponibili, tempi, costi, prestazioni e necessità di riutilizzare codice.

Un'app destinata a un dispositivo specializzato può richiedere accesso hardware nativo; un servizio informativo, invece, può privilegiare portabilità e distribuzione tramite browser.

5.2 Sviluppo nativo e multipiattaforma

Sviluppare per una piattaforma significa decidere **quali servizi utilizzare** e **quali dipendenze introdurre**. Con l'approccio **nativo** si usano linguaggi, API e strumenti specifici del sistema: per esempio UIKit/SwiftUI nell'ecosistema Apple o le API Android. La compatibilità con le convenzioni del dispositivo e l'accesso alle funzionalità di sistema risultano spesso diretti, ma la realizzazione di versioni per più piattaforme può richiedere lavoro duplicato.

L'approccio **multipiattaforma** cerca di condividere parti del codice senza mantenere implementazioni completamente distinte. Il vantaggio non è soltanto scrivere meno righe: **correggere un comportamento una volta** può ridurre i costi di manutenzione. In compenso occorre introdurre un livello di astrazione, con compromessi su prestazioni, comportamento grafico, compatibilità con nuove API e integrazione con l'hardware.

Gli strumenti multipiattaforma non funzionano tutti allo stesso modo: **React Native** usa componenti e meccanismi di integrazione con le piattaforme native; **.NET MAUI** adotta un modello di UI multipiattaforma nell'ecosistema .NET; **Flutter** utilizza widget e un motore grafico per disegnare l'interfaccia, oltre a strumenti di interoperabilità con la piattaforma (si veda anche la sezione 12). Le differenze contano quando si decide quanto controllo avere sull'aspetto della UI e su quali funzionalità del dispositivo sono disponibili.

Immaginiamo di distribuire la nostra app di note su Android e iOS. La **logica di ricerca e ordinamento** delle note è un buon candidato al riuso; autorizzazioni, notifiche, navigazione e accesso a servizi specifici potrebbero invece richiedere adattamenti. La questione da verificare non è dunque soltanto *quale framework funziona su entrambi i sistemi*, ma **quali parti restano realmente condivise senza impoverire l'esperienza d'uso**.

5.3 Il problema delle dipendenze nell'installazione tradizionale

Finora abbiamo descritto come un programma viene eseguito, ma per raggiungere un utente deve anche essere **distribuito e installato**. Un'applicazione può dipendere da codice eseguibile, librerie, runtime, configurazioni e risorse. Distribuire solo il file principale non basta se gli elementi da cui dipende sono assenti o incompatibili.

Nei modelli tradizionali l'installazione poteva comportare la copia di file in molte directory e modifiche a configurazioni condivise. Se l'applicazione A installava una certa versione di libreria e l'applicazione B ne richiedeva un'altra, aggiornare o rimuovere la libreria poteva rompere uno dei due programmi. La disinstallazione poteva inoltre lasciare file e impostazioni, perché il sistema non disponeva di una descrizione completa dei componenti coinvolti.

Queste difficoltà spiegano il valore del **packaging**: invece di distribuire una raccolta informale di file, si prepara un **pacchetto applicativo** con metadati e struttura dichiarata. Il sistema o il gestore dei pacchetti può così verificare requisiti, identificare l'app, gestire aggiornamenti e rimozioni e applicare le politiche della piattaforma.

Il pacchetto non risolve magicamente tutti i problemi: alcune dipendenze restano esterne, i dati utente possono sopravvivere all'eliminazione dell'app e le procedure cambiano tra sistemi. È però un cambiamento architetturale importante: **installazione e rimozione diventano operazioni governate da informazioni esplicite**, non soltanto azioni arbitrarie eseguite da un installer.

5.4 Pacchetti, manifest e permessi

Il **manifest** è un file di metadati che descrive al sistema operativo informazioni necessarie a riconoscere e gestire l'applicazione. A seconda della piattaforma può dichiarare identità, componenti, requisiti, versioni, funzionalità richieste e permessi. È quindi una **dichiarazione del contratto dell'app con la piattaforma**, non soltanto una lista di file.

Alcuni esempi sono:

- **Android**: `AndroidManifest.xml`, che dichiara componenti dell'app e altre informazioni rilevanti per l'esecuzione e la sicurezza;
- **Apple**: `Info.plist`, con proprietà e configurazioni del bundle; autorizzazioni ed entitlement possono richiedere anche altri meccanismi dedicati;
- **UWP**: `Package.appxmanifest`, con identità, componenti e capacità del pacchetto.

Per esempio, un'app Android che vuole usare la fotocamera può dover dichiarare il relativo permesso e, quando previsto, **chiedere all'utente l'autorizzazione durante l'uso**. La dichiarazione nel manifest e il consenso effettivo non coincidono. L'app deve anche essere progettata per il caso in cui l'utente neghi l'accesso.

La **forma del pacchetto** varia: `.apk` è un formato di pacchetto Android, `.appxbundle` è un bundle Windows, `.deb` è un pacchetto del mondo Debian. Queste estensioni non implicano lo stesso modello di isolamento e permessi. Inoltre, **pacchettizzazione e sandboxing sono proprietà distinte**: la prima riguarda la distribuzione e la gestione dell'app, il secondo le risorse che può utilizzare durante l'esecuzione.

5.5 Sandboxing e modello dei permessi

Consentire a un'applicazione di accedere indistintamente a ogni risorsa espone l'utente e il sistema a rischi, soprattutto quando l'app contiene bug o elabora dati non attendibili. Il **sandboxing** consiste nel limitarne il contesto di esecuzione: file accessibili, dispositivi,

comunicazioni e operazioni consentite. Il sistema impone un confine e controlla le richieste che lo attraversano.

Il principio generale è quello del **minimo privilegio**: accordare soltanto le capacità necessarie alla funzione richiesta. Per esempio, un'app di note testuali non dovrebbe avere bisogno della posizione precisa dell'utente; se aggiungiamo una funzione di scansione documenti, può avere motivo di accedere alla fotocamera, ma non necessariamente in ogni momento.

Occorre distinguere tre aspetti: **dichiarazione** della capacità (come nel manifest), **concessione** della capacità secondo le regole e le eventuali richieste all'utente, e **applicazione tecnica** dei limiti tramite processi isolati, permessi, API controllate e altri meccanismi. Un manifest che dichiara un permesso non garantisce da solo che l'accesso sia disponibile né che sia sicuro.

Il sandboxing riduce le conseguenze di un errore, ma **non assicura l'assenza di vulnerabilità**. Rende più difficile passare dal malfunzionamento di una singola app al controllo di altre risorse, e spiega perché i sistemi mobili trattino con particolare attenzione accesso ai dati e autorizzazioni.

Capitolo 6

Il ciclo di vita di processi e applicazioni

6.1 Perché è necessario distinguere i due cicli di vita

La parola **ciclo di vita** (*lifecycle*) indica gli stati attraversati da un'entità e le condizioni che provocano il passaggio da uno stato all'altro. Tuttavia, nel software la stessa parola viene usata per entità diverse: **processo**, **applicazione** e **componente della UI**. Sono modelli collegati, ma non sovrapponibili.

1. Il **processo del sistema operativo** può attendere la CPU, eseguire istruzioni o restare bloccato in attesa di un evento. È un modello di **pianificazione**.
2. L'**applicazione** può essere visibile all'utente, operare in background, essere sospesa o non avere più un processo attivo. È un modello di **esperienza d'uso e gestione delle risorse**.
3. Un **componente applicativo**, come una Activity Android, attraversa fasi di creazione, visibilità, interazione e distruzione. È un modello di **responsabilità del codice del componente**.

Immaginiamo che un utente stia scrivendo una nota sul telefono e apra la fotocamera. La schermata delle note smette di essere la principale; il processo che la contiene potrebbe continuare a esistere oppure essere sospeso e, in condizioni di pressione sulla memoria, terminato dal sistema. Questo non significa che la *nota stessa* debba sparire: l'applicazione deve poter ricostruire una situazione coerente al ritorno.

Il problema centrale dei sistemi mobili diventa quindi: **come mantenere continua l'esperienza percepita dall'utente quando l'esecuzione concreta non è continua?** A questa domanda non risponde la sola macchina a stati di un processo Unix: occorre anche comprendere gli stati dell'applicazione, i componenti e la persistenza dei dati.

6.2 Dal modello DOS ai sistemi multiprocesso

Nel modello tradizionale di **MS-DOS**, l'utilizzo tipico è di tipo **single-task**: il programma avviato occupa l'attenzione della macchina fino al termine o alla cessione del controllo. Nell'ambiente DOS classico in modalità reale non sono presenti la protezione della memoria e l'isolamento dei processi tipici dei sistemi moderni. La quantità di **memoria convenzionale** storicamente utilizzabile dalle applicazioni DOS è associata al limite di **640 KiB**; si tratta del modello classico, non di un limite assoluto di ogni estensione o ambiente DOS successivo.

Il software poteva interagire in modo molto diretto con l'hardware e disponeva di scarse garanzie d'isolamento. Nei sistemi moderni, invece, **multiprogrammazione**, **protezione della memoria** e **scheduler** permettono a più processi di avanzare apparentemente insieme condividendo la CPU, o realmente in parallelo quando sono disponibili più unità di esecuzione.

Anche il controllo della durata dell'esecuzione cambia: non spetta più soltanto al programma decidere *quando* può usare la CPU o quali risorse trattenere.

6.3 Gli stati fondamentali di un processo

In un sistema multiprogrammato, più processi condividono le risorse della macchina. Avere un programma pronto non significa che la CPU stia già eseguendo le sue istruzioni: lo

scheduler decide quale attività può avanzare e per quanto tempo. Il modello didattico dei processi distingue cinque stati; come chiarito in precedenza, **nei sistemi moderni questi stati descrivono più precisamente la schedulazione dei thread**, che sono le entità effettivamente assegnate ai processori. Resta utile rappresentarli attraverso un processo con un solo thread per comprendere le transizioni fondamentali:

- **New**: il processo è in fase di creazione;
- **Ready**: potrebbe eseguire, ma sta aspettando che gli venga assegnata la CPU;
- **Running**: una CPU sta effettivamente eseguendo le sue istruzioni;
- **Waiting** (o *Blocked*): non può proseguire finché non si verifica un evento, ad esempio la fine di un'operazione di input/output;
- **Terminated**: l'esecuzione è conclusa.

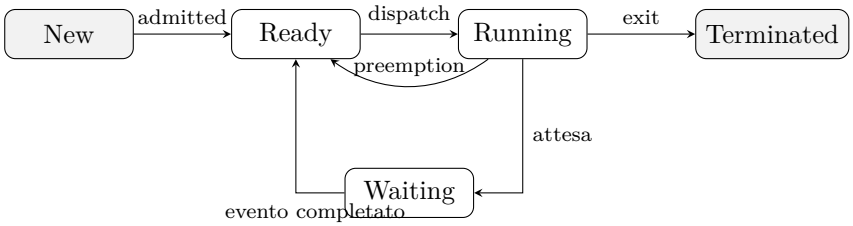


Figura 3: Stati principali di un processo e transizioni tipiche nel modello didattico dei sistemi operativi.

Esempio. Un processo che deve leggere un dato dal disco passa da *Running* a *Waiting* mentre attende l'I/O. Quando il dato è disponibile passa a *Ready*: **non rientra automaticamente in Running**, perché lo scheduler deve ancora assegnargli una CPU. Al contrario, un processo *Ready* non attende informazioni esterne, ma soltanto la disponibilità del processore. Questa differenza spiega perché *Ready* e *Waiting* non possano essere uniti in un generico stato di "non esecuzione".

Le transizioni *Ready* → *Running* e *Running* → *Ready* sono associate rispettivamente al **dispatch** e alla **preemption**: lo scheduler assegna la CPU e può successivamente sottrarla per eseguire altro. È un modello astratto utile per comprendere la gestione della CPU; i sistemi reali possono avere più code, più stati e più thread.

6.4 Creazione e sostituzione dei processi su Unix

Nella famiglia Unix, la creazione e l'esecuzione sono tradizionalmente espresse mediante funzioni distinte.

- `fork()` crea un nuovo processo figlio a partire dal processo chiamante. Il figlio ha un PID distinto. Se la chiamata ha successo, nel **padre** restituisce il PID del figlio, mentre nel **figlio** restituisce `0`; in caso di errore restituisce `-1` al chiamante.
- `exec()` indica una famiglia di funzioni che sostituiscono l'immagine del programma eseguito dal processo corrente. Normalmente il PID non cambia: cambia il programma eseguito *dentro lo stesso processo*.
- `exit()` termina il processo e produce uno stato d'uscita.
- `wait()` e `waitpid()` permettono al padre di attendere un cambiamento di stato di uno o più figli e raccoglierne le informazioni di terminazione.

L'esempio seguente mostra perché `fork()` ed `exec()` svolgano compiti diversi:

```
pid_t pid = fork();

if (pid == 0) {                  // Processo figlio
    execl("/bin/ls", "ls", "-l", (char *)NULL);
    _exit(127);                 // Solo se execl fallisce
} else if (pid > 0) {           // Processo padre
    int status;
    waitpid(pid, &status, 0);
}
```

Il padre crea un figlio con `fork()`. Il figlio prova a sostituire il proprio programma con `ls` mediante `execl()`. Se la sostituzione riesce, `execl()` non torna al codice che la segue; il padre attende la terminazione del figlio con `waitpid()`. L'esempio omette alcune verifiche degli errori per mettere in evidenza la sequenza concettuale.

6.5 Processi zombie e orfani

La fine dell'esecuzione di un processo figlio non coincide sempre con la rimozione immediata di ogni sua informazione dal sistema.

Un processo **zombie** è un processo **già terminato** del quale rimane una voce necessaria a comunicare al padre lo stato d'uscita. Il padre non ha ancora effettuato una raccolta appropriata tramite `wait()` o `waitpid()`. Lo zombie non è un programma che continua a eseguire codice: conserva soltanto alcune informazioni amministrative.

Un processo **orfano** è invece un processo il cui padre termina **prima di lui**. Il figlio può essere ancora in esecuzione e viene adottato da un processo di sistema designato a tale scopo, secondo il modello del sistema operativo (ad esempio PID 1 o un *subreaper* in Linux). Non bisogna confondere un *orfano vivo* con uno *zombie già terminato*.

6.6 Processi iniziali e servizi di sistema

Nei sistemi Unix tradizionali il processo con **PID 1**, detto `init`, svolge un ruolo essenziale nell'avvio del sistema e nella gestione o adozione dei processi. Nei Linux moderni questa funzione può essere svolta da altri sistemi di inizializzazione, come `systemd`. Dire che *tutti gli altri processi sono figli diretti di init* sarebbe scorretto: molti sono figli, nipoti o ulteriori discendenti, e la gerarchia evolve nel tempo.

In **macOS**, `launchd` svolge funzioni di inizializzazione e gestione di servizi. In **Windows**, `smss.exe` (*Session Manager Subsystem*) partecipa alle fasi iniziali di avvio delle sessioni; componenti come `winlogon.exe` gestiscono successivamente aspetti dell'accesso e della sessione interattiva. Questi esempi mostrano che l'avvio e la gestione dei processi dipendono dall'architettura del sistema operativo, senza cambiare la distinzione generale tra programma e processo.

6.7 Applicazioni desktop e ciclo degli eventi

Un'applicazione desktop grafica non si limita a produrre un output e terminare. Dopo l'inizializzazione apre finestre e rimane pronta a reagire a click, pressione di tasti, timer e richieste del sistema. Questo comportamento è spesso organizzato intorno a un **event loop**, cioè un ciclo che riceve eventi e ne attiva la gestione.

Si consideri il pulsante "Salva" dell'applicazione di note. Il sistema rileva l'interazione, il framework la trasforma in un evento e il gestore associato richiede alla logica di memorizzare il documento. Se la memorizzazione richiede una operazione lenta, la UI non dovrebbe

aspettare in modo bloccante sul proprio thread: deve rimanere disponibile per ridisegnare finestre e rispondere all'utente. Ecco perché la **programmazione asincrona** introdotta parlando di WinRT è utile anche al di fuori della piattaforma Microsoft.

Un ciclo di esecuzione desktop tipico comprende quindi avvio, inizializzazione, creazione delle finestre, gestione ripetuta degli eventi e chiusura. Non va confuso con gli stati *Ready*, *Running* e *Waiting* del processo: **una finestra aperta e apparentemente ferma può appartenere a un processo che alterna stati di attesa e brevi esecuzioni molte volte al secondo**.

Capitolo 7

Il modello di esecuzione delle applicazioni mobili

7.1 Perché il mobile richiede un modello diverso

Un computer desktop può lasciare molti programmi in esecuzione senza che l'utente presti attenzione a ciascuno. Uno smartphone deve invece conservare **reattività, memoria disponibile e autonomia della batteria**, anche quando l'utente passa rapidamente da un'app all'altra. Il sistema operativo è quindi incentivato a sospendere o terminare processi non indispensabili, per rendere disponibili risorse a quelli che l'utente sta utilizzando.

Questo non significa che gli smartphone siano computer **single-task**: possono eseguire molte attività e servizi. Significa che il sistema **controlla più strettamente il lavoro in background** e attribuisce priorità diverse alle attività in base alla loro utilità immediata e alle regole della piattaforma.

Per lo sviluppatore la conseguenza è una regola progettuale: non produrre un'applicazione che funziona soltanto se il suo processo rimane sempre vivo. Se il dispositivo deve recuperare memoria, **la ricostruzione dell'esperienza deve essere possibile a partire da uno stato salvato**. Il sistema gestisce le risorse, ma l'applicazione è responsabile della coerenza dei dati e della UI.

7.2 Foreground, background, sospensione e terminazione

Le piattaforme mobili distinguono in generale la rilevanza visiva dell'applicazione dal fatto che il suo codice stia effettivamente avanzando. Quattro concetti ricorrono spesso:

- **Foreground**: l'app è in primo piano ed è, in tutto o in parte, il contesto principale di interazione;
- **Background**: l'app non è in primo piano e può eseguire soltanto le attività consentite dal modello della piattaforma;
- **Suspended**: la normale esecuzione è sospesa, anche se parte dello stato può rimanere in memoria;
- **Not running / Terminated**: il processo non è disponibile e un nuovo accesso può richiedere l'avvio e la ricostruzione del contesto.

Background e suspended non sono sinonimi. Un'app in background potrebbe avere ancora un tempo limitato per completare un trasferimento o ricevere determinati eventi; una sospesa non esegue normalmente il proprio codice. Anche **sospensione e terminazione differiscono**: dopo la sospensione il sistema può riprendere l'esecuzione con la memoria ancora intatta; dopo la morte del processo non si possono usare gli oggetti in memoria della vecchia istanza.

Queste sono categorie per confrontare le piattaforme, **non nomi di stati universalmente identici**. iOS, UWP e Android hanno transizioni e callback specifiche, che devono essere studiate nel rispettivo contesto.

7.3 Ciclo di vita dell'applicazione su iOS

Nel modello UIKit tradizionale si incontrano gli stati seguenti:

- **Not Running**: l'applicazione non è in esecuzione;
- **Inactive**: è in foreground ma temporaneamente non riceve eventi dell'utente, per esempio durante alcune transizioni;
- **Active**: è in foreground ed è pronta a gestire normalmente gli eventi;
- **Background**: ha lasciato il foreground; alcune operazioni possono proseguire nei limiti consentiti;
- **Suspended**: resta eventualmente in memoria, senza proseguire la normale esecuzione del codice.

All'avvio una sequenza tipica è *Not Running* → *Inactive* → *Active*. Quando l'utente passa a un'altra app, si possono attraversare *Inactive*, *Background* e poi *Suspended*. Se il sistema recupera memoria, il processo può essere terminato. Le transizioni dipendono comunque dalla situazione e dalle politiche della piattaforma: non sono un percorso obbligatorio universale.

Esempio. Nell'app di note, l'arrivo di una chiamata può interrompere temporaneamente l'interazione. Se l'utente passa ad un'altra app, il sistema può sospendere quella delle note. Quando l'utente ritorna, l'app potrebbe riprendere dalla memoria oppure essere riavviata. La schermata deve in entrambi i casi ritrovare il contenuto e il contesto di lavoro necessari.

Nei sistemi Apple attuali può essere rilevante il **lifecycle delle scene** (istanze di UI) oltre a quello globale dell'app. Non bisogna assumere che ogni dispositivo supporti sempre una sola finestra o una sola scena. Il principio resta distinguere **lo stato dell'interfaccia** dalla **durata concreta del processo**.

7.4 Ciclo di vita delle applicazioni UWP

Anche **UWP** utilizza un modello di lifecycle gestito dal sistema. Un'app può trovarsi in foreground, eseguire attività di background consentite, essere **sospesa** e successivamente **ripresa** oppure **terminata** per recuperare risorse. Questo modello limita l'idea tradizionale del programma desktop che resta indefinitamente attivo solo perché l'utente non lo ha chiuso.

Un passaggio importante è la **notifica di sospensione**: quando disponibile, offre all'app una finestra limitata per prepararsi, salvare il necessario e rilasciare opportunamente le risorse. Se il processo sospeso viene in seguito terminato, non si deve fare affidamento su una seconda notifica di "ultimo istante". Il progetto corretto prevede quindi di salvare lo stato prima di potersi trovare in una condizione da cui il sistema può eliminarlo.

L'analogia con iOS è utile per capire **il problema comune**, non per affermare che stati, eventi e API siano identici. In entrambi i casi il sistema operativo può gestire la vita del processo in modo autonomo; lo sviluppatore deve rendere il ripristino coerente con l'esperienza dell'utente.

7.5 Android: componenti dell'applicazione e processi

Android rende il lifecycle più articolato perché un'applicazione è **composta da componenti** attivabili dal sistema in circostanze differenti. Le fotografie della lezione richiamano tre categorie:

- **Activity**: un punto di interazione con l'utente, normalmente associato a una schermata o una porzione significativa della UI;

- **Service**: un componente per lavoro che non ha una propria interfaccia grafica. **Non significa** automaticamente thread separato né diritto di esecuzione illimitata in background: normalmente le sue callback sono eseguite sul thread principale del processo, perciò i lavori bloccanti devono essere gestiti separatamente;
- **Broadcast Receiver**: riceve eventi o messaggi diffusi dal sistema o da altre componenti secondo le regole della piattaforma.

La piattaforma prevede anche i **Content Provider**, che espongono e gestiscono dati tramite un'interfaccia strutturata. Il punto da comprendere, più che memorizzare una lista, è che **un'app non coincide con una sola schermata**. Una app di note può avere una Activity per l'editor e usare altri componenti per reagire a eventi o gestire servizi; l'utente li percepisce come parti di un unico prodotto.

La struttura introduce due livelli distinti. Il **processo Linux** contiene codice e risorse in esecuzione; le **Activity e gli altri componenti Android** sono entità gestite secondo i contratti del framework. Quando un componente deve eseguire codice, Android può avviare il processo necessario; quando l'app perde importanza e servono risorse, il sistema può eliminarlo. Componenti diversi possono trovarsi in stati diversi pur appartenendo alla stessa applicazione.

Il lifecycle di un'Activity, che descriveremo ora, **non è quindi il lifecycle del processo Android**: è un contratto usato dal sistema per comunicare i cambiamenti che riguardano una specifica istanza della UI.

7.6 Ciclo di vita di una Activity Android

Una `Activity` riceve callback che rappresentano cambiamenti della sua visibilità e interazione. Le principali sono:

- `onCreate()`: inizializzazione dell'Activity e predisposizione del suo stato e dell'interfaccia;
- `onStart()`: l'Activity diventa visibile;
- `onResume()`: entra nello stato di interazione in primo piano;
- `onPause()`: perde lo stato di piena interazione, pur potendo essere ancora parzialmente visibile;
- `onStop()`: non è più visibile;
- `onRestart()`: precede `onStart()` quando un'Activity precedentemente fermata torna visibile;
- `onDestroy()`: callback associata alla distruzione dell'istanza dell'Activity, ad esempio per conclusione o ricreazione dovuta a cambi di configurazione.

Ad esempio, aprendo un'altra Activity, quella corrente può ricevere `onPause()` e, quando non è più visibile, `onStop()`. Tornando indietro può passare attraverso `onRestart()`, `onStart()` e `onResume()`. Se invece il sistema ha eliminato il processo mentre l'Activity era fermata, il ritorno richiede una **nuova istanza** e un ripristino dello stato.

Non si può fare affidamento su `onDestroy()` per salvare l'ultimo stato: il sistema può terminare il processo senza chiamare quella callback. Inoltre *perdita del focus*, *invisibilità*, *distruzione dell'Activity* e *terminazione del processo* sono eventi distinti, anche quando possono verificarsi in successione.

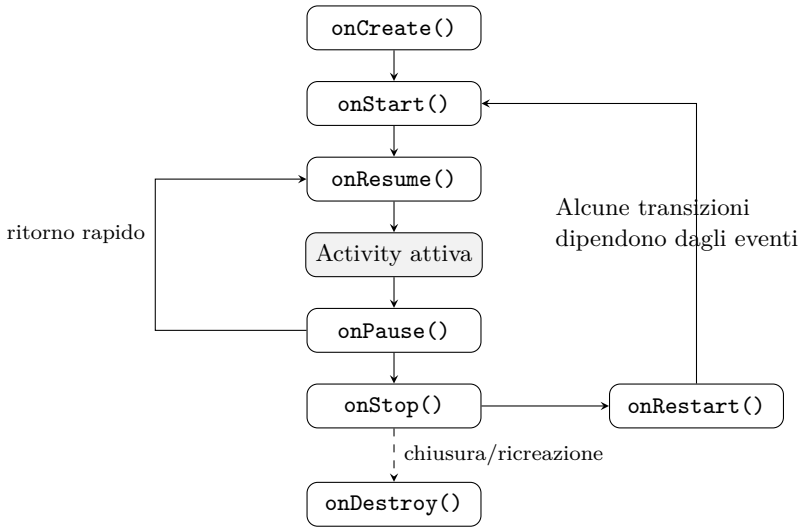


Figura 4: Transizioni principali di una Activity Android. La freccia tratteggiata riguarda la distruzione dell'istanza, non la terminazione arbitraria del processo: in quest'ultimo caso `onDestroy()` non è garantita.

7.7 Salvataggio e ripristino dello stato

Il lifecycle mobile è importante soprattutto quando dobbiamo decidere **che cosa sopravvive** all'interruzione dell'esecuzione. Nella pratica conviene distinguere almeno tre categorie di stato:

- **Dati persistenti**: note salvate, documenti, preferenze e informazioni che devono restare disponibili anche dopo la terminazione dell'app. Devono essere conservati in archivi appropriati, locali o remoti.
- **Stato temporaneo della UI**: nota selezionata, posizione di scorrimento, testo non ancora confermato, scheda aperta. Va ripristinato, quando necessario, tramite i meccanismi di stato forniti dal framework o tramite persistenza esplicita.
- **Risorse d'esecuzione**: oggetti in memoria, thread, socket e operazioni in corso. Non si può assumere che sopravvivano alla terminazione del processo; spesso vanno ricreati o riavviati in modo sicuro.

Su Android questa distinzione si traduce in meccanismi con garanzie diverse. Un `ViewModel` può conservare dati di UI in memoria quando l'Activity viene ricreata, ad esempio dopo una rotazione dello schermo, ma **non sopravvive alla terminazione del processo**. Il salvataggio dello **stato di istanza** (per esempio tramite `onSaveInstanceState()` o `SavedStateHandle`) consente di recuperare piccole informazioni transitorie, come l'identificativo della nota aperta o il testo di una ricerca, quando il sistema ricrea la schermata dopo averla distrutta. Questo stato è legato al lifecycle del componente: non sostituisce l'archiviazione permanente e non va considerato disponibile dopo qualunque modalità di chiusura.

Un archivio persistente, come un database locale o un servizio remoto, è invece necessario per le **informazioni che non devono andare perdute**, comprese le modifiche che l'utente considera già acquisite. Questi strumenti non sono alternative intercambiabili: memoria, stato di istanza e persistenza duratura risolvono problemi diversi.

Esempio completo. Un utente modifica una nota e poi passa all'app della fotocamera. Il sistema sospende l'app delle note e, sotto pressione di memoria, ne termina il processo.

Al ritorno, il processo precedente non esiste più; l'app deve leggere la nota dall'archivio e ricostruire lo stato necessario per presentare un editor coerente. Se il testo modificato non era ancora stato salvato, serve una strategia esplicita per non perderlo, come un salvataggio progressivo o la conservazione dello stato temporaneo nei limiti consentiti dalla piattaforma.

Le piattaforme offrono callback e meccanismi specifici, ma **non è corretto contare esclusivamente sull'evento di terminazione**: la fine del processo può verificarsi senza che il codice dell'app riceva un'ultima possibilità di salvare. In Android, in particolare, `onDestroy()` non è una garanzia di notifica prima della morte del processo.

Questo porta alla relazione concettuale più importante:

$$\boxed{\text{Continuità percepita dall'utente}} \neq \boxed{\text{Processo sempre in esecuzione}}.$$

La continuità dipende dal **modello di stato**, dalla **persistenza** e dalle regole del framework, non dalla permanenza illimitata di un processo in memoria.

Capitolo 8

Dall'elaborazione differita alle interfacce interattive

8.1 Perché studiare la storia delle interfacce

Un'interfaccia non è soltanto l'aspetto grafico di un'applicazione: definisce **come l'utente comunica un'intenzione al sistema, riceve una risposta e corregge il proprio comportamento**. L'evoluzione delle interfacce è quindi una storia di riduzione della distanza tra intenzione dell'utente e risultato visibile.

Il passaggio chiave è quello da un'interazione **differita**, in cui si prepara un lavoro e si attende il risultato, a un'interazione **diretta e incrementale**, in cui ogni azione può produrre rapidamente un feedback.

8.2 Schede perforate e programmazione senza feedback immediato

Nell'informatica dei primi decenni era frequente usare **schede perforate** (*punch cards*) per rappresentare dati e istruzioni. Le macchine perforatrici (*keypunch*) trasformavano la digitazione in perforazioni leggibili dal sistema. Un esempio è il modello **IBM 029**, introdotto nel **1964**: la tecnica delle schede perforate era già utilizzata molti anni prima.

Il programmatore preparava il mazzo di schede contenente istruzioni e dati, lo consegnava per l'elaborazione e riceveva il risultato soltanto successivamente. Una scheda errata poteva richiedere la correzione o sostituzione fisica e una nuova elaborazione.

In questa fase non manca ogni forma di interfaccia, ma manca la **UI interattiva in tempo reale** alla quale siamo abituati: l'input e l'output sono sostanzialmente separati nel tempo.

8.3 Batch processing: più lavori, poca interazione

Con il **batch processing** più lavori vengono raccolti e inviati insieme, in un *batch*. Il sistema esegue la sequenza secondo la propria organizzazione, senza richiedere che l'utente controlli continuamente ogni operazione.

Questo permette di utilizzare più efficientemente risorse costose, riducendo parte dei tempi morti tra un lavoro e l'altro. Dal punto di vista dell'utente, però, il feedback può arrivare dopo minuti, ore o anche intervalli più lunghi, a seconda del sistema e della coda: un errore identificato solo dopo l'esecuzione implica un nuovo ciclo di preparazione e attesa.

La differenza rispetto ai programmi moderni non è soltanto tecnologica, ma riguarda il **modo di lavorare**: anziché procedere per tentativi con correzioni immediate, si prepara il lavoro nel suo complesso e se ne verifica il risultato alla fine.

8.4 Console, teletype e interazione a comandi

L'introduzione delle **console** e dei terminali **teletype** consente di comunicare con un sistema attraverso comandi testuali. L'utente immette una richiesta, il computer la elabora e produce una risposta testuale. Si afferma così un ciclo più rapido:

$$\text{comando} \longrightarrow \text{esecuzione} \longrightarrow \text{risposta} \longrightarrow \text{nuovo comando}.$$

Su una *telescrivente* l'output poteva essere stampato su carta: il contenuto già stampato rimaneva consultabile, ma non modificabile sul supporto come avviene in un moderno editor a schermo. Il sistema può quindi diventare **esplorativo**: si tenta un comando, si osserva la risposta e si corregge la richiesta successiva.

La **Command Line Interface** (CLI) rende disponibili operazioni potenti e composibili. Richiede però all'utente di conoscere sintassi, opzioni e nomi dei comandi, con un **carico mnemonico** spesso elevato.

8.5 Time sharing e terminali video

Il **time sharing** permette a più utenti di interagire con un calcolatore condiviso grazie all'alternanza rapida dell'uso della CPU. Si passa progressivamente dall'idea di *presentare un lavoro e tornare più tardi* all'idea di *avere una sessione interattiva*.

I sistemi Unix, sviluppati a partire dalla fine degli **anni Sessanta**, rappresentano un esempio influente dell'ambiente interattivo a comandi; non nascono nel 1979, anche se versioni importanti sono state pubblicate in quel periodo.

Negli anni Settanta si diffondono i **terminali video**, che usano uno schermo al posto della stampa continua su carta. Rispetto alla teletype diventa possibile riscrivere una zona dello schermo, gestire il cursore e aggiornare la visualizzazione senza produrre ogni volta un nuovo foglio. Non tutti i primi terminali fornivano le medesime capacità, ma il cambiamento è profondo: si può progettare software nel quale l'output visibile viene **aggiornato dinamicamente**.

In questo passaggio si distinguono meglio due idee che resteranno fondamentali anche nelle interfacce moderne: **tempo di risposta** del sistema e **feedback** offerto all'utente.

8.6 Che cosa cambia per l'utente: feedback e carico cognitivo

L'evoluzione da schede perforate e batch a terminali interattivi non riguarda soltanto l'hardware di input/output. Cambia la **struttura del dialogo** tra l'utente e il sistema.

Nel batch, l'utente deve pianificare in anticipo il lavoro e attende per capire se le istruzioni sono corrette. Nella CLI può provare un comando, leggere il risultato e correggersi. La latenza più bassa favorisce quindi una strategia **esplorativa**: ogni risposta del sistema aiuta a formulare l'azione successiva.

Rimane però un limite: la riga di comando richiede di ricordare i nomi delle operazioni e la loro sintassi. Un'interfaccia che presenta visivamente oggetti, azioni e possibilità di interazione può ridurre parte di questo **carico mnemonico**, permettendo all'utente di riconoscere opzioni invece di richiamarle tutte dalla memoria. Non significa che una GUI sia sempre superiore a una CLI: per attività ripetitive e automatizzabili, una linea di comando può essere più precisa ed efficiente.

Questa tensione tra **rapidità del feedback**, **capacità dell'interfaccia** e **costo cognitivo** collega la storia delle interfacce ai criteri moderni di progettazione della UI. Non stiamo memorizzando nomi di tecnologie isolate: stiamo osservando come cambia il problema di rendere un sistema utilizzabile.

8.7 Engelbart e la "Mother of All Demos"

Il **9 dicembre 1968**, **Douglas Engelbart** e il suo gruppo presentarono il sistema **NLS** (*oN-Line System*) in una dimostrazione divenuta celebre come *Mother of All Demos*.

La presentazione mostrava idee che sarebbero diventate centrali per l'informatica interattiva: **mouse**, navigazione mediante collegamenti ipertestuali, finestre e viste su documenti, elaborazione testuale, lavoro collaborativo a distanza e comunicazione audio/video.

Il valore di questa dimostrazione non consiste nel sostenere che tutte le moderne GUI fossero già state realizzate in forma definitiva. Essa rendeva visibile un principio diverso: il computer può diventare uno **strumento interattivo per organizzare, esplorare e modificare informazioni**, invece di essere soltanto un elaboratore al quale affidare lavori preconfezionati.

8.8 Dalle prime GUI al paradigma WIMP

La dimostrazione di Engelbart anticipa la possibilità di manipolare informazioni in modo visivo. Negli anni successivi alcuni sistemi rendono questo approccio più concreto e, infine, commercialmente utilizzabile. Il **Xerox Alto** (1973) è una delle tappe decisive: impiega uno schermo grafico, un mouse e un ambiente nel quale documenti e programmi possono essere rappresentati e manipolati mediante elementi visivi. È un prototipo di ricerca, non ancora un personal computer destinato al mercato di massa.

La famiglia di interfacce che si svilupperà da queste esperienze viene spesso descritta attraverso il paradigma **WIMP**: *Windows, Icons, Menus, Pointer*, ovvero **finestre, icone, menu e puntatore**. Non è una semplice lista di decorazioni grafiche. Ognuno degli elementi risponde a un problema d'interazione: le finestre consentono di organizzare più attività visibili; le icone associano oggetti e comandi a rappresentazioni riconoscibili; i menu rendono esplorabili le azioni disponibili; il puntatore permette di selezionare e manipolare direttamente ciò che è mostrato. La differenza rispetto alla CLI non è l'assenza di comandi, ma il fatto che molte possibilità sono **presentate anziché ricordate**.

Lo **Xerox Star** (1981) porta numerose idee della ricerca Xerox in una workstation commerciale e valorizza la **metafora della scrivania** (*desktop metaphor*): documenti, cartelle e strumenti virtuali richiamano oggetti e operazioni dell'ufficio. L'**Apple Lisa** (1983) rappresenta un'altra tappa nella diffusione delle interfacce grafiche e dell'interazione tramite mouse, finestre e documenti. Il successivo Macintosh (1984) contribuirà a rendere riconoscibile questo modello a un pubblico più ampio. Non è necessario supporre che un unico prodotto abbia "inventato" l'intera GUI: il risultato deriva da una successione di ricerche, prototipi e scelte progettuali.

8.9 Metafora della scrivania e modello WYSIWYG

La metafora della scrivania è utile perché collega operazioni nuove ad azioni familiari: aprire un documento, spostarlo in una cartella, gettarlo nel cestino. Questo trasferimento di conoscenze riduce l'apprendimento iniziale, ma non è letterale: un documento digitale può essere duplicato, cercato e condiviso in modi impossibili con la carta. Una buona metafora aiuta a iniziare, senza impedire di sfruttare le possibilità del mezzo digitale.

Un'idea collegata è **WYSIWYG** (*What You See Is What You Get*): in un editor visuale l'utente modifica una rappresentazione molto vicina all'aspetto del documento finale. In un word processor può, per esempio, vedere direttamente paragrafi, formattazione e immagini mentre li modifica. L'approccio opposto non consiste necessariamente in un'interfaccia testuale: nei sistemi basati sulla **struttura semantica**, come LaTeX, l'utente esprime che una porzione di testo è una sezione, una formula o una citazione, mentre il motore di composizione ne decide la presentazione. Il termine *WYSIWYM* (*What You See Is What You Mean*) viene usato per alcuni editor orientati alla struttura; non è però un'etichetta formale che descriva automaticamente ogni editor LaTeX. Il confronto fondamentale è **controllo diretto dell'aspetto** contro **separazione fra contenuto logico e presentazione**.

8.10 X Window System: la GUI non coincide con il computer locale

Un altro sviluppo importante è l'**X Window System**, nato al MIT nel **1984** e ampiamente utilizzato negli ambienti Unix e Unix-like. Il suo contributo architetturale è la **separazione client–server** della visualizzazione grafica. Un'applicazione, detta *client X*, invia richieste a un *server X*, che gestisce le risorse del display e i dispositivi di input. Il nome può sembrare controintuitivo: il *server* si trova tipicamente sulla macchina che possiede lo schermo e serve applicazioni che possono essere eseguite anche su altri computer.

Questa divisione consente la **trasparenza rispetto alla rete**: un programma può essere eseguito su una macchina e mostrarne l'interfaccia su un'altra. Inoltre il protocollo non è legato a un particolare modello di scheda grafica. È quindi più corretto parlare di un sistema pensato per **eterogeneità hardware e accesso remoto**, non di una piattaforma "hardware dependent". Non bisogna infine identificare il protocollo X con l'intero aspetto del desktop: componenti come window manager e ambienti desktop definiscono molte delle regole di disposizione e interazione delle finestre.

8.11 Verso le GUI moderne

Il percorso storico può essere sintetizzato attraverso il problema principale risolto da ciascun passaggio:

Modello	**Cambiamento nell'interazione**
Schede perforate	Preparazione fisica dell'input, risultato differito.
Batch	Organizzazione di più lavori, ma feedback non immediato.
Console e CLI	Dialogo rapido tramite comandi e risposte.
Time sharing	Sessioni interattive di più utenti su risorse condivise.
Terminale video	Output aggiornabile sullo schermo, interazione più flessibile.
Sistemi grafici interattivi	Manipolazione visiva di oggetti, documenti e strumenti.

La GUI moderna non sostituisce in ogni contesto la CLI, che rimane utile per automazione e controllo preciso. Offre però un modello in cui molte operazioni possono essere **riconosciute visivamente** anziché richiedere la memorizzazione preventiva dei comandi. Per comprenderne davvero la progettazione dovremo studiare successivamente **finestre, eventi, elementi grafici, layout, feedback e principi di usabilità**.

Capitolo 9

Linguaggi visivi delle interfacce grafiche

Un'interfaccia deve comunicare non soltanto *che cosa contiene*, ma anche *che cosa si può fare*, *quale elemento è attivo* e *che cosa è successo dopo un'azione*. Colori, rilievi, tipografia, animazioni e trasparenze diventano quindi parte del **linguaggio di interazione**: possono migliorare la leggibilità o, se impiegati male, nascondere funzioni essenziali. Alcune tendenze storiche permettono di osservare come cambi l'equilibrio fra familiarità, semplicità e chiarezza.

9.1 Skeuomorfismo: riconoscere oggetti familiari

Lo **skeuomorfismo** impiega caratteristiche visive che richiamano un oggetto o materiale fisico anche quando la sua funzione originale non è necessaria nell'interfaccia digitale. Un blocco per prendere note può sembrare fatto di carta; un pulsante può avere rilievo e ombra come un tasto meccanico. Texture, riflessi, profondità e dettagli realistici costituiscono parte del suo vocabolario.

Il vantaggio è soprattutto la **familiarità**: l'utente può inferire come agire trasferendo esperienze del mondo reale a un ambiente nuovo. Il limite è che l'imitazione può aggiungere rumore visivo e vincolare artificialmente operazioni digitali che potrebbero essere più semplici.

L'interfaccia **Aqua** di **Mac OS X Cheetah** (2001), con pulsanti lucidi, ombre e trasparenze, è un esempio storico di forte uso di indizi materiali e tridimensionali. Non ogni ombra è però automaticamente skeuomorfismo: può anche essere un puro indizio funzionale di gerarchia.

9.2 Flat design: essenzialità e riconoscibilità delle azioni

Con il **flat design**, divenuto particolarmente influente nei primi anni Dieci, vengono ridotti rilievi, texture e imitazioni letterali di materiali. La comunicazione visiva si affida maggiormente a **griglie, tipografia, colore, spaziatura e gerarchia**. Si possono riconoscere ascendenze nel modernismo e nell'*International Typographic Style*, che valorizzano ordine, chiarezza e composizione tipografica.

L'interfaccia di **Windows 8** (2012) è un esempio evidente di questo orientamento: grandi superfici tipografiche e contenuti assumono un ruolo centrale. La semplificazione, tuttavia, introduce un compromesso. Se pulsanti, etichette e semplici testi sembrano identici, può diminuire la **riconoscibilità degli elementi azionabili** (*affordance percepita* e *signifier*). Per questo il flat design efficace non elimina ogni indicatore di interazione: usa contrasto, posizione, stato, tipografia e feedback in modo deliberato.

9.3 Material Design e il ruolo del movimento

Material Design, introdotto da Google nel 2014, combina la semplificazione grafica con un insieme coerente di regole di comportamento. Il suo "materiale" non intende riprodurre fedelmente una sostanza fisica: è una **metafora progettuale astratta** che rende comprensibili superfici, elevazione, organizzazione spaziale e transizioni.

Il **movimento** (*motion*) non serve soltanto a decorare. Se un pannello emerge da un pulsante o una scheda si sposta verso una nuova posizione, l'animazione può comunicare

continuità causale: l'utente capisce quale azione ha provocato la trasformazione e dove si trova ora l'oggetto. Un pulsante che cambia aspetto dopo l'interazione può indicare uno stato, ma il cambiamento dovrebbe essere comprensibile e non limitarsi a un effetto estetico.

Il movimento va impiegato con moderazione: animazioni lente o non necessarie ostacolano la reattività; inoltre la progettazione accessibile deve considerare le preferenze degli utenti che chiedono **movimento ridotto**. Il criterio è rendere gli stati più leggibili, non aumentare il numero di effetti.

9.4 Fluent Design, glassmorfismo e Liquid Glass

Il linguaggio **Fluent Design** di Microsoft nasce prima di Windows 11, e viene ulteriormente sviluppato in quest'ultimo sistema (2021). Accosta superfici, profondità, luce, tipografia e motion secondo regole comuni. Due materiali riconoscibili nelle interfacce Windows moderne sono **Acrylic**, che crea una superficie traslucida con effetto di sfocatura, e **Mica**, che fa partecipare il colore dello sfondo alla resa di alcune superfici dell'interfaccia. Non sono equivalenti: rispondono a esigenze differenti di gerarchia visiva e composizione.

Con il termine **glassmorfismo** si indica una tendenza progettuale che richiama pannelli di vetro attraverso traslucenza, sfocatura e bordi luminosi; **macOS Big Sur** (2020) offre esempi di uso di queste caratteristiche. In **macOS Tahoe** (2025), Apple introduce il linguaggio **Liquid Glass**, che valorizza superfici dinamiche, rifrazione, risposta alla luce e transizioni. Sono esempi utili a vedere come alcuni indizi materiali possano tornare dopo il flat design senza riproporre semplicemente il realismo fotografico delle prime GUI.

Il criterio per valutare questi linguaggi non è quale sia il più recente, ma se **contrasto, leggibilità, coerenza e possibilità di riconoscere le azioni** rimangano adeguati anche quando si sovrappongono contenuti, sfondi e trasparenze.

Capitolo 10

Rasterizzazione, densità e dimensionamento delle interfacce

Una GUI viene descritta mediante elementi come testi, icone, superfici e controlli, ma deve infine essere **rappresentata su un dispositivo fisico**. Progettare una finestra larga un certo numero di pixel non basta per stabilire quanto risulterà grande o leggibile: due schermi possono avere lo stesso numero di pixel e dimensioni fisiche differenti, oppure dimensioni simili e risoluzioni molto diverse. Per questo è necessario collegare tre livelli distinti: **rappresentazione dell'immagine**, **caratteristiche del display** e **unità di misura del layout**.

10.1 Dal tubo catodico ai pixel fisici degli schermi moderni

Il termine **pixel** (*picture element*) indica, in generale, un elemento discreto di una rappresentazione raster. Il suo significato fisico, però, dipende dalla tecnologia del display. Nei vecchi **schermi a tubo catodico** (*Cathode Ray Tube*, CRT) un fascio di elettroni percorreva lo schermo secondo righe di scansione, mentre il segnale ne modulava l'intensità luminosa. Il numero delle righe e la temporizzazione della scansione erano vincoli concreti; in senso orizzontale, invece, il segnale analogico non corrispondeva necessariamente a una matrice immutabile di celle quadrate. Un computer poteva comunque generare il segnale a partire da un *framebuffer* digitale formato da pixel: dire che il CRT "non aveva pixel" significa quindi che non aveva, in generale, la stessa griglia di *pixel fisicamente indirizzabili* dei moderni pannelli LCD, non che le immagini digitali visualizzate fossero prive di campioni.

Conviene precisare che in grafica digitale la parola pixel può indicare almeno due cose differenti: l'**image pixel** è un **campione memorizzato** in un'immagine o in un buffer; il **device pixel** è un elemento fisico indirizzabile del display. Quest'ultimo può comprendere più **subpixel**, cioè componenti luminose (spesso associate a rosso, verde e blu) la cui disposizione varia tra tecnologie e pannelli. Non c'è pertanto una corrispondenza necessaria fra un campione dell'immagine, una cella fisica e una singola componente luminosa.

Questa differenza aiuta a comprendere anche i **pixel non quadrati**: una modalità grafica da 320×200 campioni, se adattata a un'immagine visualizzata con proporzioni 4:3, non associa a ogni campione una regione quadrata. Il rapporto tra le dimensioni fisiche orizzontale e verticale di un campione dipende dal rapporto d'aspetto con il quale l'immagine viene visualizzata, non soltanto dalla risoluzione numerica.

Nei **pannelli LCD**, invece, l'immagine viene visualizzata su una **matrice fissa di celle fisiche**, ciascuna costituita da componenti che regolano la luce. Si ha quindi una **risoluzione nativa**, per esempio 1920×1080, nella quale una corrispondenza uno-a-uno fra campioni dell'immagine e celle del pannello produce normalmente il risultato più nitido. Visualizzare una risoluzione inferiore richiede un'operazione di **scaling e ricampionamento**: il pannello non cambia il numero delle proprie celle, ma il contenuto deve essere adattato alla griglia disponibile.

10.2 Rasterizzazione, aliasing e antialiasing

Una **immagine raster** è rappresentata da una matrice di campioni; una **rappresentazione vettoriale** descrive invece forme, segmenti, curve e riempimenti mediante dati geometrici. Il contenuto vettoriale può essere rasterizzato alla risoluzione di uscita richiesta, mentre l'ingrandimento di una bitmap già campionata non ricostruisce automaticamente i dettagli mancanti. Questo è uno dei motivi per cui icone e caratteri tipografici vengono spesso conservati in forma vettoriale fino al momento del rendering.

La **rasterizzazione** trasforma una descrizione geometrica, per esempio il contorno vettoriale di una lettera, in campioni di una griglia raster. La difficoltà è che il bordo ideale di una figura è continuo, mentre la griglia è discreta. Una curva può attraversare soltanto una piccola parte di un pixel: come rappresentarla usando un singolo colore per quella cella?

Una strategia elementare decide se il pixel appartiene oppure no alla figura attraverso una soglia. È una decisione binaria che può produrre **aliasing spaziale**, riconoscibile nei bordi diagonali a scalini (*jaggies*). Il problema non deriva da un difetto della curva originale: nasce dal **campionamento insufficiente** di un segnale che contiene variazioni più fini di quanto la griglia riesca a rappresentare. Può manifestarsi anche nei dettagli ripetuti e nei motivi molto fitti.

L'**antialiasing** cerca di rappresentare meglio il contributo della figura all'interno di ciascun campione. Un modello intuitivo per le superfici opache consiste nel calcolare la frazione $c \in [0,1]$ di area coperta dalla figura e mescolare il colore della figura con quello dello sfondo:

$$C_{\text{pixel}} = c\, C_{\text{figura}} + (1-c)\, C_{\text{sfondo}}.$$

Se una figura nera copre il 25% di un pixel bianco, il campione può essere rappresentato con un grigio chiaro anziché con nero o bianco puri. Al bordo di una lettera si ottiene così una transizione visiva più regolare. È un modello semplificato: la rasterizzazione reale può impiegare filtraggio, più campioni per pixel e trattamenti del colore che richiedono particolare attenzione.

Antialiasing e maggiore risoluzione non sono la stessa cosa. Aumentare i campioni disponibili riduce la dimensione dei dettagli che non si riescono a rappresentare; l'antialiasing, invece, migliora il modo in cui il contenuto continuo viene *campionato* sulla griglia esistente. Entrambi aiutano a produrre curve, testo e icone leggibili.

10.3 Dal punto tipografico alla densità dello schermo

L'esigenza di dimensionare il testo precede l'informatica. Nella storia della tipografia si sono susseguiti diversi sistemi di misura: agli studi geometrici delle lettere, fra cui quelli di Francesco Torniello da Novara nei primi decenni del Cinquecento, seguirono tentativi di standardizzare le dimensioni dei caratteri. Nel Settecento, in particolare, **Pierre-Simon Fournier** propose un sistema basato sul **punto tipografico** (1737). Le varie tradizioni tipografiche non assegnavano inizialmente allo stesso punto una lunghezza unica.

Nel *desktop publishing* e in PostScript il punto è definito come

$$1\,\text{pt}_{\text{DTP}} = \frac{1}{72}\,\text{in} \simeq 0{,}353\,\text{mm}, \qquad 1\,\text{in} = 25{,}4\,\text{mm}.$$

La misura in punti di un carattere fa riferimento al **corpo tipografico nominale**, non all'altezza esatta della singola lettera visibile. Due font da 12 punti possono avere lettere percepite come più grandi o più piccole a causa di proporzioni e metriche differenti. Inoltre il punto tipografico DTP non va confuso con il *point* logico impiegato dalle moderne API

grafiche Apple. Anche il punto interno di TeX (`pt`, pari a 1/72,27 di pollice) differisce leggermente dal punto PostScript (`bp` in TeX).

Una scelta storica che collega tipografia e interfacce digitali è l'uso della convenzione di circa **72 pixel per pollice** nei primi sistemi Macintosh: il rapporto nominale fra punti tipografici e pixel era pensato come 1:1. Windows ha invece utilizzato **96 DPI come riferimento logico**, per cui una dimensione di 1 punto tipografico DTP corrisponde nominalmente a $96/72 = 4/3$ pixel logici. Sono *convenzioni di layout*, non misurazioni sempre corrispondenti alle dimensioni reali del monitor.

Per descrivere uno schermo si usa la **densità di pixel**, misurata in **PPI** (*Pixels Per Inch*), cioè pixel fisici per pollice *di lunghezza*. La densità orizzontale, per esempio, è

$$\mathrm{PPI}_x = \frac{N_x}{L_x},$$

dove N_x è il numero di pixel fisici su una riga e L_x la larghezza fisica del pannello in pollici. Conoscendo la diagonale fisica D e la risoluzione $N_x \times N_y$ di un display con pixel quadrati si può anche calcolare

$$\mathrm{PPI} = \frac{\sqrt{N_x^2 + N_y^2}}{D}.$$

DPI (*Dots Per Inch*) misura invece, propriamente, la densità dei punti prodotti da un dispositivo di stampa. Nel software si incontra spesso il termine DPI anche per schermi e impostazioni di scaling; in questo contesto può indicare una *densità di riferimento software*, non la densità fisica esatta del pannello. Perciò DPI e PPI sono correlati nel discorso sulle interfacce, ma non sono sinonimi rigorosi.

La densità influenza direttamente la **dimensione fisica** di un contenuto espresso in pixel fisici. Se un pulsante occupa 100 pixel orizzontali, la sua larghezza sul pannello è

$$L = \frac{100}{\mathrm{PPI}} \cdot 25{,}4\,\mathrm{mm}.$$

Densità fisica	Larghezza in pixel	Larghezza effettiva
96 PPI	100 px	26,5 mm
220 PPI	100 px	11,5 mm
450 PPI	100 px	5,6 mm

Su uno schermo molto denso, dunque, un'interfaccia progettata con **dimensioni fisse in pixel fisici** può risultare minuscola pur essendo visualizzata su un pannello tecnicamente migliore. La densità da sola non determina neppure la **leggibilità percepita**: conta la distanza di osservazione (in genere diversa per telefono, portatile e monitor), oltre alle capacità visive e alle preferenze dell'utente.

10.4 Pixel logici, fattore di scala e indipendenza dal dispositivo

Per sganciare le dimensioni dell'interfaccia dalla risoluzione fisica, i sistemi grafici moderni usano **unità logiche**, spesso chiamate *device-independent pixels* (DIP) o *density-independent pixels* (dp). Queste unità descrivono il **layout** con una scala stabile per gli sviluppatori; il sistema traduce poi le misure in pixel fisici in funzione della piattaforma e delle impostazioni del display. Il termine "pixel" in queste unità è storico: **un pixel logico non coincide necessariamente con una cella fisica dello schermo**.

Indicando con s il fattore di scala, il modello di base è

$$N_{\text{fisici}} \simeq N_{\text{logici}} \cdot s.$$

Con $s = 1$ un controllo largo 44 unità logiche usa circa 44 pixel fisici; con $s = 2$ ne usa circa 88. L'uguaglianza è approssimata per possibili arrotondamenti, ricampionamenti e comportamenti specifici del rendering. Non significa che il controllo avrà *esattamente* la stessa dimensione apparente su ogni schermo: la scala dipende anche dalle convenzioni della piattaforma, dalle preferenze dell'utente e dalla distanza di visione.

Questo passaggio ha motivazioni sia grafiche sia funzionali. Il testo può essere disegnato sfruttando la maggiore risoluzione fisica senza diventare proporzionalmente più piccolo; un pulsante può mantenere una **dimensione sufficientemente comoda da riconoscere e toccare**; le icone raster possono essere offerte in varianti di densità superiore, mentre quelle vettoriali possono essere rasterizzate alla risoluzione richiesta. È quindi necessario distinguere **scaling del layout** da **qualità della rasterizzazione**: adattare la dimensione di un controllo non garantisce automaticamente che una bitmap a bassa risoluzione resti nitida.

Le piattaforme impiegano nomi e riferimenti diversi per esprimere questo principio:

Piattaforma	**Unità del layout**	**Conversione e particolarità**
Web / CSS	`px` CSS	`devicePixelRatio` indica il rapporto tra pixel fisici e pixel CSS nel contesto del browser; per definizione CSS 1 in = 96 px, non necessariamente un pollice reale sul display.
Apple / UIKit	*point* (`pt`)	Il fattore di scala del display può associare, per esempio, 1 punto logico a 2×2 o 3×3 pixel fisici.
Android	`dp`, `sp` per il testo	1 dp corrisponde approssimativamente a 1 px alla densità di riferimento di 160 dpi; `sp` considera anche la preferenza relativa alle dimensioni dei font.
Windows	DIP	La densità logica di riferimento è 96 DPI; con scaling al 150% il fattore è 1,5 rispetto alla base.
Flutter	pixel logico	`devicePixelRatio` indica quanti pixel del dispositivo corrispondono a un pixel logico nell'ambiente corrente.

Questi riferimenti non vanno equiparati a misurazioni fisiche perfette: sono **convenzioni di coordinate** definite dai rispettivi ambienti grafici. Il rapporto fra le unità si capisce osservando che un sistema può avere 1920 pixel fisici di larghezza ma presentare allo sviluppatore un'area logica di 960 unità se il fattore di scala è 2.

10.5 DPI awareness: chi gestisce l'adattamento della UI?

L'introduzione delle unità logiche solleva un'altra domanda: **chi si occupa di adattare effettivamente l'applicazione quando cambia la densità o il fattore di scala?** Nelle piattaforme desktop questo è particolarmente importante quando una finestra viene spostata tra due monitor con impostazioni differenti. Windows distingue diversi livelli di *DPI awareness*, cioè di consapevolezza dell'applicazione rispetto allo scaling.

- **DPI unaware.** L'applicazione si comporta come se operasse sempre alla densità logica di riferimento (tradizionalmente 96 DPI). Il sistema può ingrandire la sua *bitmap*

finale per adattarla allo schermo: la dimensione della finestra aumenta, ma testo e icone possono risultare **sfocati** perché vengono ingranditi campioni già calcolati.

- **System-DPI-aware.** L'applicazione prepara layout e rendering per la densità del sistema rilevata all'avvio della sessione o alla sua inizializzazione. Può risultare nitida con quel fattore di scala, ma quando viene trasferita su un monitor con scaling differente il sistema può ancora ricorrere all'ingrandimento della bitmap.
- **Per-monitor-DPI-aware.** L'applicazione riceve informazioni sulla densità del monitor sul quale si trova e, quando cambiano, **ricalcola e ridisegna** opportunamente il contenuto. In questo modo testo, icone e controlli possono sfruttare la nuova densità, anziché limitarsi a ingrandire l'immagine preesistente. Versioni più recenti del modello Windows estendono la gestione anche a numerosi elementi non-client dell'interfaccia.

Una distinzione importante è quella tra **ingrandire un'immagine** e **rieseguire il rendering**. Se una finestra preparata a 1× viene semplicemente portata a 2×, un bordo disegnato con un campione può risultare meno definito. Se invece l'applicazione ridisegna il bordo direttamente alla densità 2×, può impiegare un numero maggiore di campioni e ottenere un risultato più nitido. Ciò non significa che un'applicazione per-monitor-aware sia automaticamente accessibile: layout responsive, dimensioni dei bersagli interattivi, leggibilità tipografica e preferenze dell'utente rimangono responsabilità progettuali distinte.

Il legame con lo sviluppo mobile e con Flutter è diretto: **si progetta in coordinate logiche e si lascia al sistema di rendering la trasformazione verso i pixel fisici**, intervenendo quando dimensioni, orientamento, scaling o caratteristiche della finestra cambiano. È il motivo per cui un layout non dovrebbe fissare tutte le proprie misure in pixel hardware né trattare la risoluzione nominale come se fosse la dimensione utile disponibile all'utente.

Capitolo 11

Un esempio trasversale: la stessa applicazione su piattaforme diverse

Per ricapitolare le relazioni tra gli argomenti, riprendiamo la nostra applicazione di note. La funzionalità richiesta dall'utente è sempre la stessa: creare, modificare e ritrovare informazioni. Ciò che cambia è l'insieme dei servizi, degli strumenti e delle regole attraverso cui questa funzionalità viene realizzata.

Se la distribuiamo come **applicazione Web**, il browser fornisce l'ambiente di esecuzione JavaScript e le Web API; la UI può essere aggiornata tramite il DOM oppure tramite una libreria dichiarativa; i dati possono viaggiare verso un server mediante Fetch. Se la realizziamo come PWA, possiamo aggiungere installabilità ed eventuali strategie offline, rispettando però i limiti del browser.

Se la distribuiamo come **applicazione nativa o multipiattaforma**, cambiano framework, runtime, formato di pacchetto, API e politiche di sicurezza. Su Android, per esempio, una Activity presenta l'editor e attraversa un proprio ciclo di vita; il sistema può terminarne il processo in determinate condizioni. La logica di salvataggio deve quindi essere progettata pensando al recupero dello stato, non soltanto alla finestra che l'utente vede in quell'istante.

Se invece la portiamo su desktop con **Electron**, riusiamo una parte significativa delle tecnologie Web, ma distribuiamo anche un ambiente di esecuzione basato su Chromium e Node.js. Otteniamo alcune capacità desktop e un percorso di riuso, assumendo però i costi di memoria, dimensione e sicurezza legati al modello scelto.

In tutti i casi ricompaiono le stesse domande:

1. **Esecuzione**: chi interpreta o esegue il nostro codice?
2. **Astrazione**: quali API e framework permettono di accedere alle capacità della piattaforma?
3. **Interazione**: quale modello aggiorna la UI e comunica il risultato all'utente?
4. **Distribuzione e sicurezza**: che cosa viene installato e quali risorse può utilizzare?
5. **Ciclo di vita**: quali eventi possono interrompere l'esecuzione e quali informazioni devono sopravvivere?

Queste domande offrono un criterio per confrontare tecnologie che, a prima vista, possono sembrare una lunga serie di nomi indipendenti. La piattaforma non è un dettaglio che si aggiunge dopo aver scritto l'applicazione: **determina il modello di esecuzione, le possibilità di interazione e parte delle responsabilità dello sviluppatore**.

Capitolo 12

Laboratorio: introduzione a Flutter e Dart

Il laboratorio traduce i concetti discussi nella parte teorica in scelte e strumenti di sviluppo concreti. La piattaforma di riferimento è **Flutter**, mentre **Dart** è il linguaggio con il quale vengono scritti la logica applicativa e i componenti dell'interfaccia. Per studiare questa parte è utile conoscere già le basi della programmazione imperativa e orientata agli oggetti; ciò che interessa è imparare la sintassi specifica e comprenderne il comportamento, non ripetere nozioni comuni a tutti i linguaggi.

12.1 Flutter: scopo e modello di esecuzione

Flutter è un toolkit open source di Google per costruire **interfacce grafiche multipiattaforma** a partire da una base di codice largamente condivisa. Permette di creare applicazioni per Android, iOS, desktop e Web, pur con le differenze e i vincoli dei rispettivi ambienti. Il vantaggio non è l'esecuzione di uno stesso binario ovunque: è il riuso del *codice sorgente* e del modello di costruzione della UI.

Flutter comprende un **framework** scritto principalmente in Dart, un **motore grafico** e componenti che lo integrano con il sistema operativo. Invece di costruire l'intera interfaccia come insieme di controlli nativi differenti su Android e iOS, Flutter disegna gran parte della UI attraverso il proprio sistema di widget e rendering. Per funzioni specifiche del dispositivo può comunque comunicare con API e componenti della piattaforma tramite opportuni meccanismi di integrazione.

In sviluppo, Flutter utilizza un ambiente che supporta il ***hot reload***: molte modifiche al codice possono essere applicate rapidamente conservando lo stato corrente dell'applicazione. Non si deve confondere questo meccanismo con la compilazione per il rilascio. Su molte piattaforme native il codice Dart viene compilato **AOT** in codice macchina; sul Web l'esecuzione segue invece il modello della piattaforma browser. Pertanto "compilato nativamente" non significa che tutti i target utilizzino il medesimo artefatto.

Il percorso storico può essere ricordato senza trasformarlo in un elenco di versioni da memorizzare: il progetto nasce come **Sky** (2015), passa attraverso le versioni alpha e beta (2017–2018), raggiunge **Flutter 1.0** nel dicembre 2018 e si estende progressivamente ad altre piattaforme, con tappe quali Flutter 2 (2021) e Flutter 3 (2022). Per lo sviluppo pratico contano maggiormente le funzionalità del SDK e la versione effettivamente in uso.

12.2 Dart: ruolo e caratteristiche generali

Dart è un linguaggio **general purpose**, sviluppato da Google, con **tipizzazione statica**, supporto alla **programmazione orientata agli oggetti** e gestione automatica della memoria tramite **garbage collection**. La sua sintassi ricorda altri linguaggi con parentesi graffe, come Java, C# e Kotlin, ma presenta caratteristiche proprie per esprimere concisamente strutture di dati e componenti grafici.

Un programma Dart è organizzato in dichiarazioni, classi, funzioni e librerie. Le funzionalità definite in una libreria esterna o standard possono essere rese disponibili con `import`; una clausola `as` assegna un **prefisso** ai suoi simboli, evitando ambiguità fra nomi simili:

```
import 'dart:math' as math;
```

```
void main() {
  print(math.sqrt(9));  // 3.0
}
```

Una semplice funzione di ingresso che non richiede import è:

```
void main() {
  print('Ciao, Dart!');
}
```

`void` indica che la funzione non restituisce un valore, `main()` è il punto d'ingresso del programma e `print` stampa una rappresentazione testuale sulla console. Nei progetti Flutter l'applicazione grafica viene generalmente avviata da un `main()` che inizializza il framework e configura il widget radice: vedremo nel laboratorio successivo come viene costruita concretamente tale UI.

12.3 Organizzazione di un progetto Dart da console

Il primo esercizio svolto in laboratorio, denominato `language_tour`, non utilizza ancora Flutter: è un **programma Dart da console**. Questa scelta permette di isolare le caratteristiche del linguaggio prima di introdurre widget, rendering e ciclo di vita delle applicazioni grafiche. I concetti sintattici appresi qui saranno riutilizzati senza cambiamenti sostanziali nel codice Flutter.

Nel progetto fornito, il file `bin/language_tour.dart` contiene la funzione `main()` e costituisce il punto d'ingresso. Il file `pubspec.yaml` descrive il pacchetto, la versione minima o l'intervallo di versioni del SDK richiesto e le dipendenze. Il file `pubspec.lock` registra le versioni delle dipendenze risolte; `.dart_tool/` contiene invece informazioni generate dagli strumenti, non codice applicativo da studiare o modificare a mano. La dipendenza `args`, presente nel progetto, non è utilizzata nel sorgente dell'esercizio.

Una volta predisposto Dart, un progetto di questo tipo può essere eseguito dalla cartella radice con `dart run`. Questo comando avvia il programma da console, senza costruire una UI Flutter.

Dart supporta tre forme di commento, distinte per destinazione:

```
// Commento su una riga.

/* Commento
   su piu' righe. */

/// Documentazione di una funzione, classe o altro simbolo.
bool isEven(int n) => n % 2 == 0;
```

I commenti ordinari servono a chiarire intenzioni o passaggi non evidenti; quelli introdotti da `///` possono essere interpretati dagli strumenti di documentazione come commenti associati alla dichiarazione successiva. L'obiettivo non è descrivere ogni istruzione ovvia, ma registrare informazioni utili a chi leggerà o manterrà il codice.

Capitolo 13

Fondamenti del linguaggio Dart

13.1 Tipi, inferenza e assegnazione

In Dart si possono dichiarare variabili indicando esplicitamente il tipo oppure lasciandolo dedurre dall'espressione iniziale:

```
int eta = 22;
double prezzo = 12.50;
String nome = 'Ada';
bool iscritto = true;
var punteggio = 10;
```

`int`, `double`, `String` e `bool` sono tipi comuni, mentre `var` non significa "senza tipo": nell'ultimo esempio il compilatore deduce `int`. Una successiva assegnazione come `punteggio = 'dieci';` è quindi errata. Quando si vuole rinunciare consapevolmente a parte del controllo statico si può usare `dynamic`, ma questo sposta diversi errori possibili al runtime; per codice ordinario è preferibile conservare tipi espressivi.

Una **variabile** rappresenta un riferimento o un valore che può essere associato a un'altra espressione compatibile; questo non va confuso con la modificabilità dell'oggetto referenziato. La distinzione emerge soprattutto con `final` e `const`.

13.2 `var`, `final`, `const` e `late`

`var` dichiara una variabile normalmente riassegnabile; `final` permette di assegnarla **una sola volta**; `const` richiede invece un valore **costante a tempo di compilazione**:

```
var contatore = 0;
contatore = 1;              // Ammesso.

final istante = DateTime.now();
// istante = DateTime.now(); // Errore: riassegnazione.

const pi = 3.141592653589793;
```

Nel sorgente consegnato in laboratorio, un commento definisce `const` come una costante determinata a runtime: è un errore. `const` richiede un valore noto a **tempo di compilazione**, mentre `final` può ricevere anche un valore calcolato durante l'esecuzione.

`DateTime.now()` può essere assegnato a `final` perché il valore viene determinato durante l'esecuzione; non può essere usato come inizializzatore di una variabile `const`. Inoltre, `final` riguarda il **riferimento**, non rende immutabile ogni oggetto referenziato. Per esempio:

```
final numeri = <int>[1, 2];
numeri.add(3);     // Ammesso: la lista resta mutabile.
// numeri = [4];  // Errore: il riferimento e' final.
```

Con `late` si **rimanda l'inizializzazione** o si rende possibile una inizializzazione **lazy**. È utile quando la variabile non può essere inizializzata immediatamente ma verrà valorizzata prima dell'accesso:

```
late final String token;
token = 'abc123';
print(token);
```

`late` non costituisce però una prova statica del corretto ordine delle operazioni: leggere la variabile prima dell'inizializzazione causa un **errore a runtime**. L'aggiunta di `final` impedisce anche una seconda assegnazione. È quindi uno strumento da utilizzare quando il ciclo di inizializzazione è realmente controllato.

13.3 Null safety e operatori di accesso sicuro

La **null safety** di Dart distingue i tipi che possono contenere `null` da quelli che non lo ammettono. Per esempio, una variabile `String` è **non nullable** e deve possedere un valore compatibile quando viene utilizzata; una `String?` può invece rappresentare anche l'assenza di un testo.

```
String titolo = 'Appunti';
String? sottotitolo;

int? lunghezza = sottotitolo?.length;
int visualizzata = lunghezza ?? 0;
```

Qui `?.` esegue l'accesso al membro soltanto quando il ricevente non è `null`: altrimenti l'intera espressione produce `null`. L'operatore `??` utilizza il valore di destra **solo se** quello di sinistra è `null`.

L'operatore `!` esprime un'**asserzione di non-nullità**: `sottotitolo!` tratta il valore come non nullable, ma lancia un errore a runtime se questo è effettivamente `null`. Non costituisce una conversione che "ripara" il dato: quando possibile è preferibile verificare esplicitamente il valore.

```
if (sottotitolo != null) {
  print(sottotitolo.length);
}
```

La **promozione del tipo** tramite analisi del flusso permette al compilatore di riconoscere, in casi come questo, che all'interno del ramo controllato il valore non è nullo. Questa è la motivazione principale della null safety: intercettare molte operazioni pericolose prima dell'esecuzione, senza rinunciare ai casi in cui l'assenza di un valore è significativa.

13.4 Funzioni: parametri posizionali e nominati

Le funzioni Dart dichiarano il **tipo restituito** prima del nome. Quando il corpo consiste in una singola espressione che viene restituita, si può usare la sintassi abbreviata con `=>`:

```
int somma(int a, int b) {
  return a + b;
}

int sommaBreve(int a, int b) => a + b;
```

I parametri precedenti sono **posizionali obbligatori**: il significato dipende dalla loro posizione. I parametri in parentesi quadre sono invece **posizionali opzionali**, e possono avere un valore predefinito:

```
void saluta(String nome, [String prefisso = 'Ciao']) {
  print('$prefisso, $nome');
}

saluta('Ada');
saluta('Ada', 'Benvenuta');
```

I parametri **nominati** sono raccolti tra parentesi graffe; la chiamata ne specifica il nome, rendendo più leggibili le API con molte opzioni. Il modificatore `required` rende obbligatorio un parametro nominato:

```
void creaNota({required String titolo, bool privata = false}) {
  print('Nota: $titolo, privata: $privata');
}

creaNota(titolo: 'Esame');
creaNota(privata: true, titolo: 'Lezione');
```

L'ordine dei parametri nominati in una chiamata può variare, ma i loro nomi devono corrispondere alla dichiarazione. Non vanno confusi `required` e non-nullabilità: il primo riguarda la presenza dell'argomento, la seconda la tipologia dei valori accettati.

13.5 Condizioni, iterazioni ed espressioni condizionali

Nel progetto `language_tour` vengono utilizzate le principali strutture del **controllo di flusso**, già familiari a chi conosce Java. Con `if`, `else if` ed `else` si seleziona un ramo sulla base di condizioni booleane. Per esempio:

```
int year = 2026;
if (year >= 2001) {
  print('XXI secolo');
} else if (year >= 1901) {
  print('XX secolo');
} else {
  print('Periodo precedente');
}
```

L'ordine delle condizioni è significativo: raggiunto il primo ramo vero, quelli successivi non vengono valutati. Nell'esempio l'intervallo 1901–2000 identifica il XX secolo e 2001–2100 il XXI. Il programma originale usa lo stesso criterio per classificare un anno.

Il ciclo `for` è utile quando l'avanzamento è espresso da un inizializzatore, una condizione e un aggiornamento; `for-in` visita invece gli elementi di una collezione nell'ordine di iterazione:

```
for (int i = 0; i < 3; i++) {
  print(i);                 // 0, 1, 2
}
```

```
final planets = ['Jupiter', 'Mars', 'Saturn'];
for (final planet in planets) {
  print(planet);
}
```

Nel secondo ciclo `final` impedisce di riassegnare `planet` all'interno di una singola iterazione; non significa che la lista `planets` sia immutabile. `while` ripete un blocco finché la condizione rimane vera:

```
double money = 10.5;
while (money < 20) {
  money += 1;
}
print(money);               // 20.5
```

La condizione viene valutata **prima** di ogni iterazione. In questo esempio si effettuano dieci incrementi: 19.5 non soddisfa ancora la condizione di arresto, mentre 20.5 sì. Una condizione che non diventa mai falsa rischia di produrre un ciclo infinito.

L'**espressione condizionale** `c ? x : y` non è un altro tipo di ciclo: restituisce `x` se la condizione `c` è vera, altrimenti `y`. È appropriata quando occorre scegliere un valore breve, ad esempio per inserirlo in una stringa:

```
bool hasRings(String planet) => planet == 'Saturn';
final result = hasRings('Saturn') ? 'ha' : 'non ha';
print('Saturn $result anelli');
```

In questo caso `hasRings` è volutamente una funzione dimostrativa: riconosce soltanto `Saturn`, non rappresenta una classificazione astronomica completa. La forma `=>` equivale alla restituzione della singola espressione `planet == 'Saturn'`; la forma con `return`, presente anch'essa nell'esercizio, produce lo stesso risultato.

13.6 Interpolazione di stringhe e collezioni

L'**interpolazione** consente di includere direttamente il valore di una variabile in una stringa con `$nome`; per valutare una espressione si usa `${...}`:

```
String nome = 'Ada';
int a = 2, b = 3;
print('Ciao $nome, totale ${a + b}');
```

Le **collezioni** più comuni includono `List`, sequenza di elementi indicizzati, e `Map`, associazione fra chiavi e valori. Sono tipi generici, per cui è opportuno indicare i tipi degli elementi:

```
List<int> voti = [25, 28, 30];
voti.add(27);

Map<String, int> presenze = {
  'Ada': 4,
  'Bruno': 2,
};
print(presenze['Ada']);
```

Nella `Map` il risultato della ricerca tramite chiave può essere `null` quando la chiave è assente, anche se il tipo dei valori è `int`: questo spiega perché l'espressione `presenze['Ada']` ha in generale tipo `int?`.

13.7 Prima esercitazione: lettura del programma `language_tour`

Una volta chiarita la sintassi dei singoli costrutti, è utile vedere come essi cooperano in un programma effettivo. Il sorgente consegnato dal laboratorio contiene dichiarazioni al livello superiore della libreria, una funzione `main()`, diversi cicli e funzioni ausiliarie. La versione seguente conserva gli aspetti didattici dell'esercizio, ma corregge un accesso non sicuro a `null`, piccoli refusi testuali e una funzione dimostrativa sui pianeti. Non è un esempio di UI Flutter: serve a consolidare il linguaggio con cui la UI verrà poi programmata.

```
var year = 2026;
var money = 10.5;
var planets = ['Jupiter', 'Mars', 'Saturn'];

void main() {
  if (year >= 2001) {
    print('XXI secolo');
  } else if (year >= 1901) {
    print('XX secolo');
  } else {
    print('Periodo precedente');
  }

  for (int i = 0; i < 10; i++) {
    print(i);
  }

  for (final planet in planets) {
    print("$planet ${hasRings(planet) ? 'ha' : 'non ha'} anelli");
  }

  while (money < 20) {
    money += 1;
  }
  print('Saldo finale: $money');

  print(say('teacher', 'Hello', 'phone'));
  enableFlag(list: planets, hidden: true);
}

bool hasRings(String planet) =>
    planet == 'Jupiter' || planet == 'Saturn';

String say(String from, String msg,
    [String device = 'no_device']) {
  return '$msg, from $from ($device)';
}
```

```
void enableFlag({
  bool bold = false,
  bool hidden = false,
  int? count,
  List<String>? list,
}) {
  final number = count ?? 0;
  print('Conteggio: $number');
  print('bold: $bold, hidden: $hidden');

  if (list != null) {
    print('Lista vuota: ${list.isEmpty}');
  }
}
```

Le prime tre variabili sono dichiarate **a livello di libreria**, ossia fuori da `main()`. Sono quindi accessibili anche alle funzioni della stessa libreria e conservano il proprio valore tra chiamate successive, finché permane l'istanza di esecuzione pertinente. Questo è diverso dalla dichiarazione di una variabile *locale*, visibile soltanto nel proprio ambito. Usare troppo stato mutabile condiviso complica il ragionamento sugli effetti delle funzioni: negli esempi reali è spesso preferibile mantenere le variabili locali o passarle esplicitamente come argomenti.

Il programma esegue dapprima una selezione sull'anno, stampa i numeri `0`–`9`, visita i pianeti, incrementa `money` fino a `20.5` e invoca le due funzioni con parametri opzionali. `say` riceve un terzo parametro **posizionale opzionale**; `enableFlag` riceve invece gli argomenti **per nome**, perciò la chiamata `enableFlag(list: planets, hidden: true)` non deve necessariamente specificare `bold` e `count`. Nell'esempio `count` vale `null` e l'espressione `count ?? 0` fornisce il valore alternativo `0`.

È importante osservare il difetto del **sorgente originale**: subito dopo aver calcolato `count ?? 0`, la funzione esegue anche la seguente istruzione:

```
int number2 = count!;
```

Poiché la chiamata in `main()` non fornisce `count`, questa istruzione produce un'**eccezione a runtime**. Il precedente operatore `??` non ha modificato `count`: ha soltanto calcolato un altro valore, sicuro da utilizzare. L'operatore `!` invece asserisce che il valore originale non è nullo e fallisce se l'asserzione è falsa. Per questo la versione riveduta elimina tale asserzione e verifica `list != null` prima dell'accesso a `list.isEmpty`. Un'alternativa, quando interessa un valore predefinito, è utilizzare `count ?? 0` direttamente senza conservarne una seconda copia nullable.

Nel programma ricevuto compare anche `list?.isEmpty;` come espressione isolata: l'accesso è sicuro, ma il risultato viene ignorato e quindi non produce alcun effetto osservabile. Usarlo in una `print`, in una condizione o assegnarlo a una variabile rende l'operazione significativa. Analogamente, la mappa `image` dichiarata nel sorgente originale mostra la sintassi dei dizionari, ma non viene utilizzata dalla funzione `main()`.

Questo esercizio consolida una distinzione da conservare anche nello sviluppo Flutter: **il controllo statico dei tipi riduce gli errori, ma non sostituisce il ragionamento sul flusso dell'esecuzione**. Un programma può essere sintatticamente valido e ben tipizzato, ma fallire per una condizione non verificata, una risorsa mancante o un'asserzione di non-nullità ingiustificata.

13.8 Classi, costruttori e incapsulamento

Dart è orientato agli oggetti: una **classe** definisce dati e comportamenti, e un oggetto rappresenta un'istanza con il relativo stato. L'ereditarietà con `extends` permette di riusare ed estendere una superclasse; un `mixin`, utilizzato tramite `with`, consente di comporre comportamenti riutilizzabili senza costruire necessariamente una gerarchia di superclassi.

I costruttori possono utilizzare parametri come `this.nome` per inizializzare direttamente campi dell'oggetto e `super(...)` per passare i dati al costruttore della superclasse. Dart supporta anche **costruttori nominati**, utili per fornire modalità alternative di costruzione, e **costruttori `factory`**, utili quando la creazione richiede logica preliminare o può restituire istanze esistenti o di sottotipi.

Un identificatore che comincia con `_`, come `_accessi`, ha **visibilità privata a livello di libreria** Dart: non indica in generale una privacy limitata alla singola classe, come in alcuni altri linguaggi. Infine un **getter** permette di esporre una proprietà calcolata usando la sintassi di accesso a un campo, senza costringere a memorizzare un valore che dipende da altri campi.

13.9 Esempio completo: modellare uno studente

Il seguente esempio riunisce la sintassi mostrata a lezione. È pensato per essere letto dall'alto verso il basso: prima la superclasse, poi un comportamento riutilizzabile, infine la classe con costruttori, proprietà e metodo `factory`.

```
class Person {
  final String name;
  Person(this.name);
}

mixin Loggable {
  void log(String message) => print(message);
}

class Student extends Person with Loggable {
  final int studentId;
  final List<int> grades;
  String? email;
  int _accessi = 0;

  Student({
    required String name,
    required this.studentId,
    List<int>? grades,
    this.email,
  }) : grades = grades ?? <int>[], super(name);

  Student.withoutId(String name)
      : this(name: name, studentId: 0);

  factory Student.fromMap(Map<String, dynamic> data) {
    return Student(
```

```
      name: data['name'] as String,
      studentId: data['id'] as int,
      grades: List<int>.from(data['grades'] as List? ?? []),
    );
  }

  double get average => grades.isEmpty
      ? 0.0
      : grades.reduce((a, b) => a + b) / grades.length;

  void registerAccess() {
    _accessi++;
    log('Accessi: $_accessi');
  }
}

void main() {
  final student = Student(
    name: 'Ada',
    studentId: 7,
    grades: [28, 30],
  );
  student.registerAccess();
  print(student.average);
}
```

Il costruttore ordinario accetta **parametri nominati**: `name` e `studentId` sono obbligatori; `grades` ed `email` sono opzionali. Nella **lista di inizializzazione**, l'espressione `grades ?? <int>[]` crea una lista vuota quando non ne viene fornita una; `super(name)` inizializza il campo ereditato dalla classe `Person`. Il costruttore nominato `Student.withoutId` **reindirizza** a quello principale.

`Student.fromMap` è un costruttore `factory`: legge dati da una mappa e li trasforma negli argomenti richiesti. I cast `as String` e `as int` **non convalidano in modo completo i dati**: se la mappa non contiene valori dei tipi previsti, l'esecuzione può fallire. In un'applicazione reale si dovrebbero aggiungere controlli ed eventuale gestione degli errori.

Il getter `average` usa l'**operatore condizionale** `condizione ? valoreVero : valoreFalso`: evita una divisione per zero nel caso di lista vuota e, negli altri casi, somma i voti con `reduce` prima di dividerli per il numero degli elementi. `_accessi` è privato alla libreria, mentre `log` proviene dal mixin. La lista `grades`, benché associata a un campo `final`, può comunque essere modificata: per ottenere una vera immutabilità dei voti occorrerebbero ulteriori vincoli sulla collezione.

L'esempio mostra perché la sintassi non è solo un elenco di simboli: `required`, `?`, `??`, `final`, `factory` e `get` determinano **contratti concreti** su inizializzazione, presenza dei valori, costruzione e accesso allo stato. Sono gli stessi aspetti che diventeranno importanti quando oggetti e stato verranno collegati all'interfaccia Flutter.